C++ Standard Library Issues to be moved in Issaquah, Feb. 2023

Doc. no. P2789R0

Date: 2023-02-06

Audience: WG21
Reply to: Jonathan Wakely <lwgchair@gmail.com>

Ready Issues

2195. Missing constructors for `match_results`

Section: 32.9 [re.results] **Status:** Ready **Submitter:** Daniel Krügler **Opened:** 2012-10-06 **Last modified:** 2022-11-10

Priority: 3

View all other issues in [re.results].

Discussion:

The requirement expressed in 32.9 [re.results] p2

> The class template `match_results` shall satisfy the requirements of an allocator-aware container and of a sequence container, as specified in 24.2.4 [sequence.reqmts], except that only operations defined for const-qualified sequence containers are supported.

can be read to require the existence of the described constructors from as well, but they do not exist in the synopsis.

The missing sequence constructors are:

```
match_results(initializer_list<value_type>);
match_results(size_type, const value_type&);
template<class InputIterator> match_results(InputIterator, InputIterator);
```

The missing allocator-aware container constructors are:

```
match_results(const match_results&, const Allocator&);
match_results(match_results&&, const Allocator&);
```

It should be clarified, whether (a) constructors are an exception of above mentioned operations or (b) whether at least some of them (like those accepting a `match_results` value and an allocator) should be added.

As visible in several places of the standard (including the core language), constructors seem usually to be considered as "operations" and they certainly can be invoked for const-qualified objects.

The below given proposed resolution applies only the minimum necessary fix, i.e. it excludes constructors from above requirement.

[2013-04-20, Bristol]

Check current implementations to see what they do and, possibly, write a paper.

[2013-09 Chicago]

Ask Daniel to update the proposed wording to include the allocator copy and move constructors.

[2014-01-18 Daniel changes proposed resolution]

Previous resolution from Daniel [SUPERSEDED]:

> 1. Change 32.9 [re.results] p2 as indicated:
>
> > The class template `match_results` shall satisfy the requirements of an allocator-aware container and of a sequence container, as specified in 24.2.4 [sequence.reqmts], except that only operations defined for const-qualified sequence containers that are not constructors are supported.

[2015-05-06 Lenexa]

MC passes important knowledge to EF.

VV, RP: Looks good.

TK: Second form should be conditionally noexcept

JY: Sequence constructors are not here, but mentioned in the issue writeup. Why?

TK: That would have been fixed by the superseded wording.

JW: How does this interact with Mike Spertus' allocator-aware regexes? [...] Perhaps it doesn't.

JW: Can't create match_results, want both old and new resolution.

JY: It's problematic that users can't create these, but not this issue.

VV: Why conditional noexcept?

MC: Allocator move might throw.

JW: Update superseded wording to "only non-constructor operations that are"?

MC: Only keep superseded, but append "and the means of constructing match_results are limited to [...]"?

JY: Bullet 4 paragraph 2 needs to address the allocator constructor.

Assigned to JW for drafting.

[2015-10, Kona Saturday afternoon]

STL: I want Mike Spertus to be aware of this issue.

Previous resolution from Daniel [SUPERSEDED]:

This wording is relative to N3936.

1. Change 32.9 [re.results] p4, class template `match_results` synopsis, as indicated:

```
[…]
// 28.10.1, construct/copy/destroy:
explicit match_results(const Allocator& a = Allocator());
match_results(const match_results& m);
match_results(const match_results& m, const Allocator& a);
match_results(match_results&& m) noexcept;
match_results(match_results&& m, const Allocator& a) noexcept;
[…]
```

2. Change 32.9.2 [re.results.const] as indicated: [*Drafting note:* Paragraph 6 as currently written, makes not much sense, because the `noexcept` does not allow any exception to propagate. Further-on, the allocator requirements do not allow for throwing move constructors. Deleting it seems to be near to editorial — *end drafting note*]

```
match_results(const match_results& m);
match_results(const match_results& m, const Allocator& a);
```

-4- *Effects:* Constructs an object of class `match_results`, as a copy of `m`.

```
match_results(match_results&& m) noexcept;
match_results(match_results&& m, const Allocator& a) noexcept;
```

-5- *Effects:* Move-constructs an object of class `match_results` from `m` satisfying the same postconditions as Table 142. ~~Additionally~~For the first form, the stored `Allocator` value is move constructed from `m.get_allocator()`.

~~-6- *Throws:* Nothing if the allocator's move constructor throws nothing.~~

[2019-03-27 Jonathan updates proposed resolution]

Previous resolution [SUPERSEDED]:

This wording is relative to N4810.

These edits overlap with the proposed resolution of 2191 but it should be obvious how to resolve the conflicts. Both resolutions remove the word "Additionally" from p4. Issue 2191 removes the entire *Throws:* element in p5 but this issue replaces it with different text that applies to the new constructor only.

1. Change 32.9 [re.results] p4, class template `match_results` synopsis, as indicated:

```
[…]
// 30.10.1, construct/copy/destroy:
explicit match_results(const Allocator& a = Allocator());
match_results(const match_results& m);
match_results(const match_results& m, const Allocator& a);
match_results(match_results&& m) noexcept;
match_results(match_results&& m, const Allocator& a);
[…]
```

2. Change 32.9.2 [re.results.const] as indicated:

```
match_results(const match_results& m);
match_results(const match_results& m, const Allocator& a);
```

-3- *Effects:* Constructs an object of class `match_results`, as a copy of `m`. For the second form, the stored `Allocator` value is constructed from `a`.

```
match_results(match_results&& m) noexcept;
match_results(match_results&& m, const Allocator& a);
```

-4- *Effects:* Move-constructs an object of class `match_results` from `m` satisfying the same postconditions as Table 128. ~~Additionally~~For the first form, the stored `Allocator` value is move constructed from `m.get_allocator()`. For the second form, the stored `Allocator` value is constructed from `a`.

-6- *Throws:* ~~Nothing.~~ The second form throws nothing if `a == m.get_allocator()`.

[2022-11-06; Daniel syncs wording with recent working draft]

To ensure that all constructors are consistent in regard to the information about how the stored allocator is constructed, more wording is added. This harmonizes with the way how we specify the individual container constructors (Such as `vector`) even though 24.2.2.5 [container.alloc.reqmts] already provides some guarantees. For the copy-constructor we intentionally refer to 24.2.2.2 [container.reqmts] so that we don't need to repeat what is said there.

[Kona 2022-11-08; Move to Ready]

Proposed resolution:

This wording is relative to N4917.

1. Change 32.9 [re.results], class template `match_results` synopsis, as indicated:

```
[…]
// 32.9.2 [re.results.const], construct/copy/destroy:
match_results() : match_results(Allocator()) {}
explicit match_results(const Allocator& a);
match_results(const match_results& m);
match_results(const match_results& m, const Allocator& a);
match_results(match_results&& m) noexcept;
match_results(match_results&& m, const Allocator& a);
[…]
```

2. Change 32.9.2 [re.results.const] as indicated:

```
explicit match_results(const Allocator& a);
```

-?- *Effects:* The stored `Allocator` value is constructed from `a`.

-2- *Postconditions*: `ready()` returns `false`. `size()` returns `0`.

```
match_results(const match_results& m);
match_results(const match_results& m, const Allocator& a);
```

-?- *Effects:* For the first form, the stored `Allocator` value is obtained as specified in 24.2.2.2 [container.reqmts]. For the second form, the stored `Allocator` value is constructed from `a`.

-3- *Postconditions*: As specified in Table 142 [tab:re.results.const].

```
match_results(match_results&& m) noexcept;
match_results(match_results&& m, const Allocator& a);
```

-4- *Effects:* For the first form, t~~T~~he stored `Allocator` value is move constructed from `m.get_allocator()`. For the second form, the stored `Allocator` value is constructed from `a`.

-5- *Postconditions*: As specified in Table 142 [tab:re.results.const].

-?- *Throws:* The second form throws nothing if `a == m.get_allocator()` is `true`.

2295. Locale name when the provided `Facet` is a `nullptr`

Section: 30.3.1.3 [locale.cons] **Status:** Ready **Submitter:** Juan Soulie **Opened:** 2013-09-04 **Last modified:** 2022-11-09

Priority: 3

View other active issues in [locale.cons].

View all other issues in [locale.cons].

Discussion:

30.3.1.3 [locale.cons] p14 ends with:

> "[…] If `f` is null, the resulting object is a copy of `other`."

but the next line p15 says:

> "*Remarks:* The resulting locale has no name."

But both can't be true when `other` has a name and `f` is null.

I've tried it on two implementations (MSVC,GCC) and they are inconsistent with each other on this.

Daniel Krügler:

As currently written, the *Remarks* element applies unconditionally for all cases and thus should "win". The question arises whether the introduction of this element by LWG 424 had actually intended to change the previous *Note* to a *Remarks* element. In either case the wording should be improved to clarify this special case.

[2022-02-14; Daniel comments]

This issue seems to have some overlap with LWG 3676 so both should presumably be resolved in a harmonized way.

[2022-11-01; Jonathan provides wording]

This also resolves 3673 and 3676.

[2022-11-04; Jonathan revises wording after feedback]

Revert an incorrect edit to p8, which was incorrectly changed to:

"If `cats` is equal to `locale::none`, the resulting locale has the same name as `locale(std_name)`. Otherwise, the locale has a name if and only if `other` has a name."

[Kona 2022-11-08; Move to Ready status]

Proposed resolution:

This wording is relative to N4917.

1. Modify 30.3.1.3 [locale.cons] as indicated:

```
explicit locale(const char* std_name);
```

-2- *Effects*: Constructs a locale using standard C locale names, e.g., `"POSIX"`. The resulting locale implements semantics defined to be associated with that name.

-3- *Throws*: `runtime_error` if the argument is not valid, or is null.

-4- *Remarks*: The set of valid string argument values is `"C"`, `""`, and any implementation-defined values.

```
explicit locale(const string& std_name);
```

-5- *Effects*: ~~The same as~~Equivalent to `locale(std_name.c_str())`.

```
locale(const locale& other, const char* std_name, category cats);
```

-?- *Preconditions*: `cats` is a valid `category` value (30.3.1.2.1 [locale.category]).

-6- *Effects*: Constructs a locale as a copy of `other` except for the facets identified by the `category` argument, which instead implement the same semantics as `locale(std_name)`.

-7- *Throws*: `runtime_error` if the second argument is not valid, or is null.

-8- *Remarks*: The locale has a name if and only if `other` has a name.

```
locale(const locale& other, const string& std_name, category cats);
```

-9- *Effects*: ~~The same as~~Equivalent to `locale(other, std_name.c_str(), cats)`.

```
template<class Facet> locale(const locale& other, Facet* f);
```

-10- *Effects*: Constructs a locale incorporating all facets from the first argument except that of type `Facet`, and installs the second argument as the remaining facet. If `f` is null, the resulting object is a copy of `other`.

-11- *Remarks*: If `f` is null, the resulting locale has the same name as `other`. Otherwise, the ~~The~~ resulting locale has no name.

```
locale(const locale& other, const locale& one, category cats);
```

-?- *Preconditions*: `cats` is a valid `category` value.

-12- *Effects*: Constructs a locale incorporating all facets from the first argument except for those that implement `cats`, which are instead incorporated from the second argument.

-13- *Remarks*: If `cats` is equal to `locale::none`, the resulting locale has a name if and only if the first argument has a name. Otherwise, the ~~The~~ locale has a name if and only if the first two arguments both have names.

3032. `ValueSwappable` requirement missing for `push_heap` and `make_heap`

Section: 27.8.8 [alg.heap.operations] **Status:** Ready **Submitter:** Robert Douglas **Opened:** 2017-11-08 **Last modified:** 2022-11-10

Priority: 3

View all other issues in [alg.heap.operations].

Discussion:

In discussion of D0202R3 in Albuquerque, it was observed that `pop_heap` and `sort_heap` had `constexpr` removed for their requirement of `ValueSwappable`. It was then observed that `push_heap` and `make_heap` were not similarly marked as having the `ValueSwappable` requirement. The room believed this was likely a specification error, and asked to open an issue to track it.

[2017-11 Albuquerque Wednesday night issues processing]

Priority set to 3; Marshall to investigate

Previous resolution [SUPERSEDED]:

This wording is relative to N4700.

1. Change 27.8.8.2 [push.heap] as indicated:

```
template<class RandomAccessIterator>
  void push_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  void push_heap(RandomAccessIterator first, RandomAccessIterator last,
                 Compare comp);
```

-1- *Requires:* The range `[first, last - 1)` shall be a valid heap. `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (16.4.4.3 [swappable.requirements]). The type of `*first` shall satisfy the `MoveConstructible` requirements (Table 23) and the `MoveAssignable` requirements (Table 25).

2. Change 27.8.8.4 [make.heap] as indicated:

```
template<class RandomAccessIterator>
  void make_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  void make_heap(RandomAccessIterator first, RandomAccessIterator last,
                 Compare comp);
```

-1- *Requires:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (16.4.4.3 [swappable.requirements]). The type of `*first` shall satisfy the `MoveConstructible` requirements (Table 23) and the `MoveAssignable` requirements (Table 25).

[2022-11-06; Daniel comments and syncs wording with recent working draft]

For reference, the finally accepted paper was P0202R3 and the constexpr-ification of swap-related algorithms had been realized later by P0879R0 after resolution of CWG 1581 and more importantly CWG 1330.

[Kona 2022-11-09; Move to Ready]

Proposed resolution:

This wording is relative to N4917.

1. Change 27.8.8.2 [push.heap] as indicated:

```
template<class RandomAccessIterator>
  constexpr void push_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
  constexpr void push_heap(RandomAccessIterator first, RandomAccessIterator last,
                           Compare comp);

template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
         class Proj = identity>
  requires sortable<I, Comp, Proj>
  constexpr I
    ranges::push_heap(I first, S last, Comp comp = {}, Proj proj = {});
template<random_access_range R, class Comp = ranges::less, class Proj = identity>
  requires sortable<iterator_t<R>, Comp, Proj>
  constexpr borrowed_iterator_t<R>
    ranges::push_heap(R&& r, Comp comp = {}, Proj proj = {});
```

-1- Let `comp` be `less{}` and `proj` be `identity{}` for the overloads with no parameters by those names.

-2- *Preconditions*: The range `[first, last - 1)` is a valid heap with respect to `comp` and `proj`. For the overloads in namespace `std`, `RandomAccessIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3 [swappable.requirements]) and the type of `*first` meets the *Cpp17MoveConstructible* requirements (Table 32) and the *Cpp17MoveAssignable* requirements (Table 34).

-3- *Effects*: Places the value in the location `last - 1` into the resulting heap `[first, last)`.

-4- *Returns*: `last` for the overloads in namespace `ranges`.

-5- *Complexity*: At most log(`last - first`) comparisons and twice as many projections.

2. Change 27.8.8.4 [make.heap] as indicated:

```
template<class RandomAccessIterator>
  constexpr void make_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
  constexpr void make_heap(RandomAccessIterator first, RandomAccessIterator last,
                           Compare comp);

template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
         class Proj = identity>
  requires sortable<I, Comp, Proj>
  constexpr I
    ranges::make_heap(I first, S last, Comp comp = {}, Proj proj = {});
```

```
template<random_access_range R, class Comp = ranges::less, class Proj = identity>
  requires sortable<iterator_t<R>, Comp, Proj>
  constexpr borrowed_iterator_t<R>
    ranges::make_heap(R&& r, Comp comp = {}, Proj proj = {});
```

-1- Let `comp` be `less{}` and `proj` be `identity{}` for the overloads with no parameters by those names.

-2- *Preconditions*: For the overloads in namespace `std`, `RandomAccessIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3 [swappable.requirements]) and the type of `*first` meets the *Cpp17MoveConstructible* (Table 32) and *Cpp17MoveAssignable* (Table 34) requirements.

-3- *Effects*: Constructs a heap with respect to `comp` and `proj` out of the range `[first, last)`.

-4- *Returns*: `last` for the overloads in namespace `ranges`.

-5- *Complexity*: At most 3(`last - first`) comparisons and twice as many projections.

3085. `char_traits::copy` precondition too weak

Section: 23.2.2 [char.traits.require] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2018-03-16 **Last modified:** 2022-11-11

Priority: 2

View other active issues in [char.traits.require].

View all other issues in [char.traits.require].

Discussion:

Table 54, Character traits requirements, says that `char_traits::move` allows the ranges to overlap, but `char_traits::copy` requires that `p` is not in the range `[s, s + n)`. This appears to be an attempt to map to the requirements of `memmove` and `memcpy` respectively, allowing those to be used to implement the functions, however the requirements for `copy` are weaker than those for `memcpy`. The C standard says for `memcpy` "If copying takes place between objects that overlap, the behavior is undefined" which is a stronger requirement than the start of the source range not being in the destination range.

All of libstdc++, libc++ and VC++ simply use `memcpy` for `char_traits<char>::copy`, resulting in undefined behaviour in this example:

```
char p[] = "abc";
char* s = p + 1;
std::char_traits<char>::copy(s, p, 2);
assert(std::char_traits<char>::compare(p, "aab", 3) == 0);
```

If the intention is to allow `memcpy` as a valid implementation then the precondition is wrong (unfortunately nobody realized this when fixing `char_traits::move` in LWG DR 7). If the intention is to require `memmove` then it is strange to have separate `copy` and `move` functions that both use `memmove`.

N.B. `std::copy` and `std::copy_backward` are not valid implementations of `char_traits::copy` either, due to different preconditions.

Changing the precondition implicitly applies to `basic_string::copy` (23.4.3.7.7 [string.copy]), and `basic_string_view::copy` (23.3.3.8 [string.view.ops]), which are currently required to support partially overlapping ranges:

```
std::string s = "abc";
s.copy(s.data() + 1, s.length() - 1);
assert(s == "aab");
```

[2018-04-03 Priority set to 2 after discussion on the reflector.]

[2018-08-23 Batavia Issues processing]

No consensus for direction; revisit in San Diego. Status to Open.

[2022-04-25; Daniel rebases wording on N4910]

Previous resolution [SUPERSEDED]:

This wording is relative to N4910.

Option A:

1. Edit 23.2.2 [char.traits.require], Table 75 — "Character traits requirements" [tab:char.traits.req], as indicated:

Table 75 — Character traits requirements [tab:char.traits.req]

Expression	Return type	Assertion/note pre/post-condition	Complexity
[…]			
`X::copy(s,p,n)`	`X::char_type*`	*Preconditions:* ~~`p not in [s,s+n)`~~ The ranges `[p,p+n)` and `[s,s+n)` do not overlap. *Returns:* `s`. for each `i` in `[0,n)`, performs `X::assign(s[i],p[i])`.	linear
[…]			

Option B:

NAD (i.e. implementations need to be fixed, in practice `char_traits::copy` and `char_traits::move` might be equivalent).

[Kona 2022-11-11; Move to Ready]

LWG voted for Option A (6 for, 0 against, 1 netural)

Proposed resolution:

1. Edit 23.2.2 [char.traits.require], Table 75 — "Character traits requirements" [tab:char.traits.req], as indicated:

Table 75 — Character traits requirements [tab:char.traits.req]

Expression	Return type	Assertion/note pre/post-condition	Complexity
[…]			
`X::copy(s,p,n)`	`X::char_type*`	*Preconditions:* ~~`p` not in `[s,s+n)`~~ The ranges `[p,p+n)` and `[s,s+n)` do not overlap. *Returns:* `s`. for each `i` in `[0,n)`, performs `X::assign(s[i],p[i])`.	linear
[…]			

3664. LWG 3392 broke `std::ranges::distance(a, a+3)`

Section: 25.4.4.3 [range.iter.op.distance] **Status:** Ready **Submitter:** Arthur O'Dwyer **Opened:** 2022-01-23 **Last modified:** 2022-11-10

Priority: 2

View all other issues in [range.iter.op.distance].

Discussion:

Consider the use of `std::ranges::distance(first, last)` on a simple C array. This works fine with `std::distance`, but currently does not work with `std::ranges::distance`.

```
// godbolt link
#include <ranges>
#include <cassert>

int main() {
  int a[] = {1, 2, 3};
  assert(std::ranges::distance(a, a+3) == 3);
  assert(std::ranges::distance(a, a) == 0);
  assert(std::ranges::distance(a+3, a) == -3);
}
```

Before LWG 3392, we had a single iterator-pair overload:

```
template<input_or_output_iterator I, sentinel_for<I> S>
  constexpr iter_difference_t<I> distance(I first, S last);
```

which works fine for C pointers. After LWG 3392, we have two iterator-pair overloads:

```
template<input_or_output_iterator I, sentinel_for<I> S>
  requires (!sized_sentinel_for<S, I>)
    constexpr iter_difference_t<I> distance(I first, S last);

template<input_or_output_iterator I, sized_sentinel_for<I> S>
  constexpr iter_difference_t<I> distance(const I& first, const S& last);
```

and unfortunately the one we want — `distance(I first, S last)` — is no longer viable because [with `I=int*`, `S=int*`], we have `sized_sentinel_for<S, I>` and so its constraints aren't satisfied. So we look at the other overload [with `I=int[3]`, `S=int[3]`], but unfortunately its constraints aren't satisfied either, because `int[3]` is not an `input_or_output_iterator`.

[2022-01-30; Reflector poll]

Set priority to 2 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to N4901.

> [*Drafting Note:* Thanks to Casey Carter. Notice that `sentinel_for<S, I>` already implies and subsumes `input_or_output_iterator<I>`, so that constraint wasn't doing anything; personally I'd prefer to remove it for symmetry (and to save the environment). Otherwise you'll have people asking why one of the `I`'s is constrained and the other isn't.]

1. Modify 25.2 [iterator.synopsis], header `<iterator>` synopsis, as indicated:

```
[…]
// 25.4.4.3 [range.iter.op.distance], ranges::distance
template<class~~input_or_output_iterator~~ I, sentinel_for<I> S>
  requires (!sized_sentinel_for<S, I>)
```

```
  constexpr iter_difference_t<I> distance(I first, S last);
template<class~~input_or_output_iterator~~ I, sized_sentinel_for<decay_t<I>> S>
  constexpr iter_difference_t<I> distance(const I& first, ~~const~~ S~~&~~ last);
[…]
```

2. Modify 25.4.4.3 [range.iter.op.distance] as indicated:

```
template<class~~input_or_output_iterator~~ I, sentinel_for<I> S>
  requires (!sized_sentinel_for<S, I>)
  constexpr iter_difference_t<I> ranges::distance(I first, S last);
```

-1- *Preconditions:* `[first, last)` denotes a range.

-2- *Effects:* Increments `first` until `last` is reached and returns the number of increments.

```
template<class~~input_or_output_iterator~~ I, sized_sentinel_for<decay_t<I>> S>
  constexpr iter_difference_t<I> ranges::distance(const I& first, ~~const~~ S~~&~~ last);
```

-3- *Effects:* Equivalent to: `return last - first;`

[2022-02-16; Arthur and Casey provide improved wording]

[Kona 2022-11-08; Move to Ready]

Proposed resolution:

This wording is relative to N4901.

[*Drafting Note:* Arthur thinks it's a bit "cute" of the *Effects:* element to `static_cast` from `T(&)[N]` to `T* const&` in the array case, but it does seem to do the right thing in all cases, and it saves us from having to use an `if constexpr (is_array_v...)` or something like that.]

1. Modify 25.2 [iterator.synopsis], header `<iterator>` synopsis, as indicated:

```
[…]
// 25.4.4.3 [range.iter.op.distance], ranges::distance
template<class~~input_or_output_iterator~~ I, sentinel_for<I> S>
  requires (!sized_sentinel_for<S, I>)
  constexpr iter_difference_t<I> distance(I first, S last);
template<class~~input_or_output_iterator~~ I, sized_sentinel_for<decay_t<I>> S>
  constexpr iter_difference_t<decay_t<I>> distance(~~const~~ I~~&~~ first, ~~const~~ S~~&~~ last);
[…]
```

2. Modify 25.4.4.3 [range.iter.op.distance] as indicated:

```
template<class~~input_or_output_iterator~~ I, sentinel_for<I> S>
  requires (!sized_sentinel_for<S, I>)
  constexpr iter_difference_t<I> ranges::distance(I first, S last);
```

-1- *Preconditions:* `[first, last)` denotes a range.

-2- *Effects:* Increments `first` until `last` is reached and returns the number of increments.

```
template<class~~input_or_output_iterator~~ I, sized_sentinel_for<decay_t<I>> S>
  constexpr iter_difference_t<decay_t<I>> ranges::distance(~~const~~ I~~&~~ first, ~~const~~ S~~&~~ last);
```

-3- *Effects:* Equivalent to: `return last - static_cast<const decay_t<I>&>(first);`

3720. Restrict the valid types of *arg-id* for *width* and *precision* in *std-format-spec*

Section: 22.14.2.2 [format.string.std] **Status:** Ready **Submitter:** Mark de Wever **Opened:** 2022-06-19 **Last modified:** 2022-11-10

Priority: 2

View other active issues in [format.string.std].

View all other issues in [format.string.std].

Discussion:

Per 22.14.2.2 [format.string.std]/7

> If `{ arg-id`$_{opt}$ `}` is used in a *width* or *precision*, the value of the corresponding formatting argument is used in its place. If the corresponding formatting argument is not of integral type, or its value is negative for *precision* or non-positive for *width*, an exception of type `format_error` is thrown.

The issue is the integral type requirement. The following code is currently valid:

```
std::cout << std::format("{:*^{}}\n", 'a', '0');
std::cout << std::format("{:*^{}}\n", 'a', true);
```

The output of the first example depends on the value of `'0'` in the implementation. When a `char` has `signed char` as underlying type negative values are invalid, while the same value would be valid when the underlying type is `unsigned char`. For the second example the range of a boolean is very small, so this seems not really useful.

Currently libc++ rejects these two examples and MSVC STL accepts them. The members of the MSVC STL team, I spoke, agree these two cases should be rejected.

The following integral types are rejected by both libc++ and MSVC STL:

```
std::cout << std::format("{:*^{}}\n", 'a', L'0');
std::cout << std::format("{:*^{}}\n", 'a', u'0');
std::cout << std::format("{:*^{}}\n", 'a', U'0');
std::cout << std::format("{:*^{}}\n", 'a', u8'0');
```

In order to accept these character types they need to meet the basic formatter requirements per 22.14.5 [format.functions]/20 and 22.14.5 [format.functions]/25

> `formatter<remove_cvref_t<`T_i`>, charT>` meets the *BasicFormatter* requirements (22.14.6.1 [formatter.requirements]) for each T_i in `Args`.

which requires adding the following enabled formatter specializations to 22.14.6.3 [format.formatter.spec].

```
template<> struct formatter<wchar_t, char>;

template<> struct formatter<char8_t, charT>;
template<> struct formatter<char16_t, charT>;
template<> struct formatter<char32_t, charT>;
```

Note, that the specialization `template<> struct formatter<char, wchar_t>` is already required by the Standard.

Not only do they need to be added, but it also needs to be specified how they behave when their value is not in the range of representable values for `charT`.

Instead of requiring these specializations, I propose to go the other direction and limit the allowed types to signed and unsigned integers.

[2022-07-08; Reflector poll]

Set priority to 2 after reflector poll. Tim Song commented:

> "This is technically a breaking change, so we should do it sooner rather than later.
>
> "I don't agree with the second part of the argument though - I don't see how this wording requires adding those transcoding specializations. Nothing in this wording requires integral types that cannot be packed into basic_format_arg to be accepted.
>
> "I also think we need to restrict this to signed or unsigned integer types with size no greater than sizeof(long long). Larger types get type-erased into a handle and the value isn't really recoverable without heroics."

Previous resolution [SUPERSEDED]:

This wording is relative to N4910.

1. Modify 22.14.2.2 [format.string.std] as indicated:

   -7- If `{` *arg-id*$_{opt}$ `}` is used in a *width* or *precision*, the value of the corresponding formatting argument is used in its place. If the corresponding formatting argument is not of ~~integral~~signed or unsigned integer type, or its value is negative for *precision* or non-positive for *width*, an exception of type `format_error` is thrown.

2. Add a new paragraph to C.1.9 [diff.cpp20.utilities] as indicated:

   Affected subclause: 22.14 [format]

   Change: Requirement changes of *arg-id* of the *width* and *precision* fields of *std-format-spec*. *arg-id* now requires a signed or unsigned integer type instead of an integral type.

   Rationale: Avoid types that are not useful and the need to specify enabled formatter specializations for all character types.

   Effect on original feature: Valid C++ 2020 code that passes a boolean or character type as *arg-id* becomes invalid. For example:

   ```
   std::format("{:*^{}}", "", true); // ill-formed, previously returned "*"
   ```

[2022-11-01; Jonathan provides improved wording]

Previous resolution [SUPERSEDED]:

This wording is relative to N4917.

1. Modify 22.14.2.2 [format.string.std] as indicated:

   -8- If `{` *arg-id*$_{opt}$ `}` is used in a *width* or *precision*, the value of the corresponding formatting argument is used in its place. If the corresponding formatting argument is not of ~~integral~~standard signed or unsigned integer type, or its value is negative, an exception of type `format_error` is thrown.

2. Add a new paragraph to C.1.9 [diff.cpp20.utilities] as indicated:

   Affected subclause: 22.14.2.2 [format.string.std]

   Change: Restrict types of formatting arguments used as *width* or *precision* in a *std-format-spec*.

   Rationale: Avoid types that are not useful or do not have portable semantics.

   Effect on original feature: Valid C++ 2020 code that passes a boolean or character type as *arg-id* becomes invalid. For example:

```
std::format("{:*^{}}", "", true); // ill-formed, previously returned "*"
std::format("{:*^{}}", "", '1'); // ill-formed, previously returned an implementation-defined number of '*' characters
```

[2022-11-10; Jonathan revises wording]

Improve Annex C entry.

[Kona 2022-11-10; Move to Ready]

Proposed resolution:

This wording is relative to N4917.

1. Modify 22.14.2.2 [format.string.std] as indicated:

   -8- If `{ arg-id`$_{opt}$ `}` is used in a *width* or *precision*, the value of the corresponding formatting argument is used in its place. If the corresponding formatting argument is not of ~~integral~~standard signed or unsigned integer type, or its value is negative, an exception of type `format_error` is thrown.

2. Add a new paragraph to C.1.9 [diff.cpp20.utilities] as indicated:

   Affected subclause: 22.14.2.2 [format.string.std]

   Change: Restrict types of formatting arguments used as *width* or *precision* in a *std-format-spec*.

   Rationale: Disallow types that do not have useful or portable semantics as a formatting width or precision.

   Effect on original feature: Valid C++ 2020 code that passes a boolean or character type as *arg-id* becomes invalid. For example:

   ```
   std::format("{:*^{}}", "", true); // ill-formed, previously returned "*"
   std::format("{:*^{}}", "", '1'); // ill-formed, previously returned an implementation-defined number of '*' characters
   ```

3756. Is the `std::atomic_flag` class signal-safe?

Section: 17.13.5 [support.signal], 33.5.10 [atomics.flag] **Status:** Ready **Submitter:** Ruslan Baratov **Opened:** 2022-08-18 **Last modified:** 2022-11-11

Priority: 3

Discussion:

Following document number N4910 about signal-safe instructions 17.13.5 [support.signal] Signal handlers, and it's unclear whether `std::atomic_flag` is signal-safe.

Formally it doesn't fit any of the mentioned conditions:

- `f` is a non-static member function invoked on an object `A`, such that `A.is_lock_free()` yields `true`, or

  > (there is no `is_lock_free` method in `std::atomic_flag` class)

- `f` is a non-member function, and for every pointer-to-atomic argument `A` passed to `f`, `atomic_is_lock_free(A)` yields `true`

  > (`std::atomic_flag` object can't be passed to `atomic_is_lock_free` as argument)

However, `std::atomic_flag` seem to fit well here, it's atomic, and it's always lock-free.

The suggestion is as follows: If `std::atomic_flag` is signal-safe, then it should be explicitly mentioned in 17.13.5 [support.signal], e.g., the following lines should be added:

- `f` is a non-static member function invoked on an `atomic_flag` object, or
- `f` is a non-member function, and every pointer-to-atomic argument passed to `f` is `atomic_flag`, or

If the `std::atomic_flag` is not signal-safe, the following note could be added:

> [*Note:* Even though `atomic_flag` is atomic and lock-free, it's not signal-safe. — *end note*]

[2022-09-23; Reflector poll]

Set priority to 3 after reflector poll. Send to SG1.

Another way to fix this is to add `is_always_lock_free (=true)` and `is_lock_free() { return true; }` to `atomic_flag`.

[Kona 2022-11-10; SG1 yields a recommendation]

Poll: Adopt the proposed resolution for LWG3756
"f is a non-static member function invoked on an atomic_flag object, or"
"f is a non-member function, and every pointer-to- atomic argument passed to f is atomic_flag, or"

```
SF F N A SA
11 3 0 0 0
```

Unanimous consent

Previous resolution [SUPERSEDED]:

This wording is relative to N4917.

1. Modify 17.13.5 [support.signal] as indicated:

-1- A call to the function `signal` synchronizes with any resulting invocation of the signal handler so installed.

-2- A *plain lock-free atomic operation* is an invocation of a function `f` from 33.5 [atomics], such that:

(2.1) — `f` is the function `atomic_is_lock_free()`, or
(2.2) — `f` is the member function `is_lock_free()`, or
(2.?) — `f` is a non-static member function invoked on an `atomic_flag` object, or
(2.?) — `f` is a non-member function, and every pointer-to-atomic argument passed to `f` is `atomic_flag`, or
(2.3) — `f` is a non-static member function invoked on an object `A`, such that `A.is_lock_free()` yields `true`, or
(2.4) — `f` is a non-member function, and for every pointer-to-atomic argument `A` passed to `f`, `atomic_is_lock_free(A)` yields `true`.

-3- An evaluation is *signal-safe* unless it includes one of the following:

(3.1) — a call to any standard library function, except for plain lock-free atomic operations and functions explicitly identified as signal-safe;

[*Note 1*: This implicitly excludes the use of `new` and `delete` expressions that rely on a library-provided memory allocator. — *end note*]

(3.2) — an access to an object with thread storage duration;
(3.3) — a `dynamic_cast` expression;
(3.4) — throwing of an exception;
(3.5) — control entering a *try-block* or *function-try-block*;
(3.6) — initialization of a variable with static storage duration requiring dynamic initialization (6.9.3.3 [basic.start.dynamic], 8.8 [stmt.dcl])[206]; or
(3.7) — waiting for the completion of the initialization of a variable with static storage duration (8.8 [stmt.dcl]).

A signal handler invocation has undefined behavior if it includes an evaluation that is not signal-safe.

[2022-11-11; Jonathan provides improved wording]

[Kona 2022-11-11; Move to Ready]

Proposed resolution:

This wording is relative to N4917.

1. Modify 17.13.5 [support.signal] as indicated:

-1- A call to the function `signal` synchronizes with any resulting invocation of the signal handler so installed.

-2- A *plain lock-free atomic operation* is an invocation of a function `f` from 33.5 [atomics], such that:

(2.1) — `f` is the function `atomic_is_lock_free()`, or
(2.2) — `f` is the member function `is_lock_free()`, or
(2.?) — `f` is a non-static member function of class `atomic_flag`, or
(2.?) — `f` is a non-member function, and the first parameter of `f` has type *cv* `atomic_flag*`, or
(2.3) — `f` is a non-static member function invoked on an object `A`, such that `A.is_lock_free()` yields `true`, or
(2.4) — `f` is a non-member function, and for every pointer-to-atomic argument `A` passed to `f`, `atomic_is_lock_free(A)` yields `true`.

-3- An evaluation is *signal-safe* unless it includes one of the following:

(3.1) — a call to any standard library function, except for plain lock-free atomic operations and functions explicitly identified as signal-safe;

[*Note 1*: This implicitly excludes the use of `new` and `delete` expressions that rely on a library-provided memory allocator. — *end note*]

(3.2) — an access to an object with thread storage duration;
(3.3) — a `dynamic_cast` expression;
(3.4) — throwing of an exception;
(3.5) — control entering a *try-block* or *function-try-block*;
(3.6) — initialization of a variable with static storage duration requiring dynamic initialization (6.9.3.3 [basic.start.dynamic], 8.8 [stmt.dcl])[206]; or
(3.7) — waiting for the completion of the initialization of a variable with static storage duration (8.8 [stmt.dcl]).

A signal handler invocation has undefined behavior if it includes an evaluation that is not signal-safe.

3769. `basic_const_iterator::operator==` causes infinite constraint recursion

Section: 25.5.3 [const.iterators] **Status:** Ready **Submitter:** Hewill Kang **Opened:** 2022-09-05 **Last modified:** 2022-11-10

Priority: 1

View other active issues in [const.iterators].

View all other issues in [const.iterators].

Discussion:

Currently, `basic_const_iterator::operator==` is defined as a `friend` function:

```
template<sentinel_for<Iterator> S>
  friend constexpr bool operator==(const basic_const_iterator& x, const S& s);
```

which only requires `S` to model `sentinel_for<Iterator>`, and since `basic_const_iterator` has a conversion constructor that accepts `I`, this will result in infinite constraint checks when comparing `basic_const_iterator<int*>` with `int*` (online example):

```
#include <iterator>

template<std::input_iterator I>
struct basic_const_iterator {
  basic_const_iterator() = default;
  basic_const_iterator(I);
  template<std::sentinel_for<I> S>
  friend bool operator==(const basic_const_iterator&, const S&);
};

static_assert(std::sentinel_for<basic_const_iterator<int*>, int*>); // infinite meta-recursion
```

That is, `sentinel_for` ends with *`weakly-equality-comparable-with`* and instantiates `operator==`, which in turn rechecks `sentinel_for` and instantiates the same `operator==`, making the circle closed.

The proposed resolution is to change `operator==` to be a member function so that `S` is no longer accidentally instantiated as `basic_const_iterator`. The same goes for `basic_const_iterator::operator-`.

[2022-09-23; Reflector poll]

Set priority to 1 after reflector poll.

"Although I am not a big fan of member ==, the proposed solution seems to be simple." "prefer if we would keep `operator==` as non-member for consistency."

Previous resolution from Hewill [SUPERSEDED]:

This wording is relative to N4917.

1. Modify 25.5.3.3 [const.iterators.iterator], class template `basic_const_iterator` synopsis, as indicated:

```
namespace std {
  template<class I>
    concept not-a-const-iterator = see below;

  template<input_iterator Iterator>
  class basic_const_iterator {
    Iterator current_ = Iterator();
    using reference = iter_const_reference_t<Iterator>;         // exposition only

  public:
    […]
    template<sentinel_for<Iterator> S>
      friend constexpr bool operator==(const basic_const_iterator& x, const S& s) const;
    […]
    template<sized_sentinel_for<Iterator> S>
      friend constexpr difference_type operator-(const basic_const_iterator& x, const S& y) const;
    template<not-a-const-iterator sized_sentinel_for<Iterator> S>
      requires sized sentinel_for different_from<S, Iterator basic_const_iterator>
      friend constexpr difference_type operator-(const S& x, const basic_const_iterator& y);
  };
}
```

2. Modify 25.5.3.5 [const.iterators.ops] as indicated:

[…]

```
  template<sentinel_for<Iterator> S>
    friend constexpr bool operator==(const basic_const_iterator& x, const S& s) const;
```

-16- *Effects*: Equivalent to: `return x.current_ == s;`.

[…]

```
  template<sized_sentinel_for<Iterator> S>
    friend constexpr difference_type operator-(const basic_const_iterator& x, const S& y) const;
```

-24- *Effects*: Equivalent to: `return x.current_ - y;`.

```
  template<not-a-const-iterator sized_sentinel_for<Iterator> S>
    requires sized sentinel_for different_from<S, Iterator basic_const_iterator>
    friend constexpr difference_type operator-(const S& x, const basic_const_iterator& y);
```

-25- *Effects*: Equivalent to: `return x - y.current_;`.

[2022-11-04; Tomasz comments and improves proposed wording]

Initially, LWG requested an investigation of alternative resolutions that would avoid using member functions for the affected operators. Later, it was found that in addition to ==/-, all comparison operators (<, >, <=, >=, <=>) are affected by same problem for the calls with `basic_const_iterator<basic_const_iterator<int*>>` and `int*` as arguments, i.e. `totally_ordered_with<basic_const_iterator<basic_const_iterator<int*>>, int*>` causes infinite recursion in constraint checking.

The new resolution, change all of the friends overloads for operators ==, <, >, <=, >=, <=> and - that accept `basic_const_iterator` as lhs, to `const` member functions. This change is applied to homogeneous `(basic_const_iterator, basic_const_iterator)` for consistency. For the overload of <, >, <=, >= and - that accepts `(I, basic_const_iterator)` we declared them as friends and consistently constrain them with *`not-const-iterator`*. Finally, its put (now member) `operator<=>(I)` in the block with other heterogeneous overloads in the synopsis.

The use of member functions addresses issues, because:

- it disallows conversion to `basic_const_iterator` in the left-hand side of op, i.e. eliminates issues for `(sized_)sentinel_for<basic_const_iterator<int*>, int*>` and `totally_ordered<basic_const_iterator<int*>, int*>`
- member functions (in contrast to friends) are not found by ADL, so we do not get multiple candidates for `basic_const_iterator<basic_const_iterator<S>>`, so we address recursion for nested iterators

[Kona 2022-11-08; Move to Ready]

Proposed resolution:

This wording is relative to N4917.

1. Modify 25.5.3.3 [const.iterators.iterator], class template `basic_const_iterator` synopsis, as indicated:

```
namespace std {
  template<class I>
    concept not-a-const-iterator = see below;

  template<input_iterator Iterator>
  class basic_const_iterator {
    Iterator current_ = Iterator();
    using reference = iter_const_reference_t<Iterator>;         // exposition only

  public:
    […]
    template<sentinel_for<Iterator> S>
      friend constexpr bool operator==(const basic_const_iterator& x, const S& s) const;

    friend constexpr bool operator<(const basic_const_iterator& x, const basic_const_iterator& y) const
      requires random_access_iterator<Iterator>;
    friend constexpr bool operator>(const basic_const_iterator& x, const basic_const_iterator& y) const
      requires random_access_iterator<Iterator>;
    friend constexpr bool operator<=(const basic_const_iterator& x, const basic_const_iterator& y) const
      requires random_access_iterator<Iterator>;
    friend constexpr bool operator>=(const basic_const_iterator& x, const basic_const_iterator& y) const
      requires random_access_iterator<Iterator>;
    friend constexpr auto operator<=>(const basic_const_iterator& x, const basic_const_iterator& y) const
      requires random_access_iterator<Iterator> && three_way_comparable<Iterator>;

    template<different-from<basic_const_iterator> I>
    friend constexpr bool operator<(const basic_const_iterator& x, const I& y) const
      requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
    template<different-from<basic_const_iterator> I>
    friend constexpr bool operator>(const basic_const_iterator& x, const I& y) const
      requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
    template<different-from<basic_const_iterator> I>
    friend constexpr bool operator<=(const basic_const_iterator& x, const I& y) const
      requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
    template<different-from<basic_const_iterator> I>
    friend constexpr bool operator>=(const basic_const_iterator& x, const I& y) const
      requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
    template<different-from<basic_const_iterator> I>
    constexpr auto operator<=>(const I& y) const
      requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I> &&
               three_way_comparable_with<Iterator, I>;
    template<not-a-const-iterator I>
    friend constexpr bool operator<(const I& y, const basic_const_iterator& x)
      requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
    template<not-a-const-iterator I>
    friend constexpr bool operator>(const I& y, const basic_const_iterator& x)
      requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
    template<not-a-const-iterator I>
    friend constexpr bool operator<=(const I& y, const basic_const_iterator& x)
      requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
    template<not-a-const-iterator I>
    friend constexpr bool operator>=(const I& y, const basic_const_iterator& x)
      requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
    template<different-from<basic_const_iterator> I>
    friend constexpr auto operator<=>(const basic_const_iterator& x, const I& y)
      requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I> &&
               three_way_comparable_with<Iterator, I>;


    […]
    template<sized_sentinel_for<Iterator> S>
      friend constexpr difference_type operator-(const basic_const_iterator& x, const S& y) const;
    template<not-a-const-iterator S>
      requires sized_sentinel_for<S, Iterator>
      friend constexpr difference_type operator-(const S& x, const basic_const_iterator& y);
  };
}
```

2. Modify 25.5.3.5 [const.iterators.ops] as indicated:

[…]

```
template<sentinel_for<Iterator> S>
  friend constexpr bool operator==(const basic_const_iterator& x, const S& s) const;
```

-16- *Effects*: Equivalent to: `return x.current_ == s;`

```
friend constexpr bool operator<(const basic_const_iterator& x, const basic_const_iterator& y) const
  requires random_access_iterator<Iterator>;
friend constexpr bool operator>(const basic_const_iterator& x, const basic_const_iterator& y) const
  requires random_access_iterator<Iterator>;
friend constexpr bool operator<=(const basic_const_iterator& x, const basic_const_iterator& y) const
  requires random_access_iterator<Iterator>;
friend constexpr bool operator>=(const basic_const_iterator& x, const basic_const_iterator& y) const
  requires random_access_iterator<Iterator>;
friend constexpr auto operator<=>(const basic_const_iterator& x, const basic_const_iterator& y) const
  requires random_access_iterator<Iterator> && three_way_comparable<Iterator>;
```

-17- Let *op* be the operator.

-18- *Effects*: Equivalent to: `return x.current_ op y.current_;`

```
template<different-from<basic_const_iterator> I>
friend constexpr bool operator<(const basic_const_iterator& x, const I& y) const
  requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<different-from<basic_const_iterator> I>
friend constexpr bool operator>(const basic_const_iterator& x, const I& y) const
  requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<different-from<basic_const_iterator> I>
friend constexpr bool operator<=(const basic_const_iterator& x, const I& y) const
  requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<different-from<basic_const_iterator> I>
friend constexpr bool operator>=(const basic_const_iterator& x, const I& y) const
  requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<different-from<basic_const_iterator> I>
friend constexpr auto operator<=>(const basic_const_iterator& x, const I& y) const
  requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I> &&
               three_way_comparable_with<Iterator, I>;
```

-19- Let *op* be the operator.

-20- ~~*Returns*~~*Effects*: Equivalent to: `return x.current_ op y;`

[…]

```
template<sized_sentinel_for<Iterator> S>
  friend constexpr difference_type operator-(const basic_const_iterator& x, const S& y) const;
```

-24- *Effects*: Equivalent to: `return x.current_ - y;`

```
template<not-a-const-iterator~~sized_sentinel_for<Iterator>~~ S>
  requires sized_sentinel_for~~different_from~~<S, Iterator~~basic_const_iterator~~>
  friend constexpr difference_type operator-(const S& x, const basic_const_iterator& y);
```

-25- *Effects*: Equivalent to: `return x - y.current_;`

3807. The feature test macro for `ranges::find_last` should be renamed

Section: 17.3.2 [version.syn] **Status:** Ready **Submitter:** Daniel Marshall **Opened:** 2022-11-02 **Last modified:** 2022-11-12

Priority: Not Prioritized

View other active issues in [version.syn].

View all other issues in [version.syn].

Discussion:

The current feature test macro is `__cpp_lib_find_last` which is inconsistent with almost all other ranges algorithms which are in the form `__cpp_lib_ranges_xxx`.

Proposed resolution is to rename the macro to `__cpp_lib_ranges_find_last`.

[Kona 2022-11-12; Move to Ready]

Proposed resolution:

This wording is relative to N4917.

1. Modify 17.3.2 [version.syn], header `<version>` synopsis, as indicated:

```
[…]
#define __cpp_lib_filesystem              201703L // also in <filesystem>
#define __cpp_lib_ranges_find_last        202207L // also in <algorithm>
#define __cpp_lib_flat_map                202207L // also in <flat_map>
[…]
```

3811. `views::as_const` on `ref_view<T>` should return `ref_view<const T>`

Section: 26.7.21.1 [range.as.const.overview] **Status:** Ready **Submitter:** Tomasz Kamiński **Opened:** 2022-11-03 **Last modified:** 2022-11-10

Priority: Not Prioritized

View other active issues in [range.as.const.overview].

View all other issues in [range.as.const.overview].

Discussion:

For `v` being a non-const lvalue of type `std::vector<int>`, `views::as_const(v)` produces `ref_view<std::vector<int> const>`. However, when `v` is converted to `ref_view` by using `views::all`, `views::as_const(views::all(v))` produces `as_const_view<ref_view<std::vector<int>>>`.

Invoking `views::as_const` on `ref_view<T>` should produce `ref_view<const T>` when `const T` models a constant range. This will reduce the number of instantiations, and make a behavior of `views::as_const` consistent on references and `ref_view` to containers.

[Kona 2022-11-08; Move to Ready]

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.7.21.1 [range.as.const.overview] as indicated:

> [*Drafting note*: If we have `ref_view<V>`, when `V` is constant propagating view (`single_view`, `owning_view`), we still can (and should) produce `ref_view<V const>`. This wording achieves that.]

-2- The name `views::as_const` denotes a range adaptor object (26.7.2 [range.adaptor.object]). Let `E` be an expression, let `T` be `decltype((E))`, and let `U` be `remove_cvref_t<T>`. The expression `views::as_const(E)` is expression-equivalent to:

(2.1) — If `views::all_t<T>` models `constant_range`, then `views::all(E)`.

(2.2) — Otherwise, if `U` denotes `span<X, Extent>` for some type `X` and some extent `Extent`, then `span<const X, Extent>(E)`.

(2.?) — Otherwise, if `U` denotes `ref_view<X>` for some type `X` and `const X` models `constant_range`, then `ref_view(static_cast<const X&>(E.base()))`.

(2.3) — Otherwise, if `E` is an lvalue, `const U` models `constant_range`, and `U` does not model `view`, then `ref_view(static_cast<const U&>(E))`.

(2.4) — Otherwise, `as_const_view(E)`.

3820. `cartesian_product_view::iterator::prev` is not quite right

Section: 99 [ranges.cartesian.iterator] **Status:** Ready **Submitter:** Hewill Kang **Opened:** 2022-11-08 **Last modified:** 2022-11-10

Priority: Not Prioritized

View other active issues in [ranges.cartesian.iterator].

View all other issues in [ranges.cartesian.iterator].

Discussion:

Currently, `cartesian_product_view::iterator::prev` has the following *Effects*:

```
auto& it = std::get<N>(current_);
if (it == ranges::begin(std::get<N>(parent_->bases_))) {
  it = cartesian-common-arg-end(std::get<N>(parent_->bases_));
  if constexpr (N > 0) {
    prev<N - 1>();
  }
}
--it;
```

which decrements the underlying iterator one by one using recursion. However, when `N == 0`, it still detects if the first iterator has reached the beginning and assigns it to the end, which is not only unnecessary, but also causes *`cartesian-common-arg-end`* to be applied to the first range, making it ill-formed in some cases, for example:

```
#include <ranges>

int main() {
  auto r = std::views::cartesian_product(std::views::iota(0));
  r.begin() += 3; // hard error
}
```

This is because, for the first range, `cartesian_product_view::iterator::operator+=` only requires it to model `random_access_range`. However, when `x` is negative, this function will call *`prev`* and indirectly calls *`cartesian-common-arg-end`*, since the latter constrains its argument to satisfy *`cartesian-product-common-arg`*, that is, `common_range<R> || (sized_range<R> && random_access_range<R>)`, which is not the case for the unbounded `iota_view`, resulting in a hard error in *`prev`*'s function body.

The proposed resolution changes the position of the `if constexpr` so that we just decrement the first iterator and nothing else.

[Kona 2022-11-08; Move to Ready]

Proposed resolution:

This wording is relative to N4917.

1. Modify 99 [ranges.cartesian.iterator] as indicated:

```
template<size_t N = sizeof...(Vs)>
  constexpr void prev();
```

-6- *Effects*: Equivalent to:

```
auto& it = std::get<N>(current_);
if constexpr (N > 0) {
  if (it == ranges::begin(std::get<N>(parent_->bases_))) {
    it = cartesian-common-arg-end(std::get<N>(parent_->bases_));
    if constexpr (N > 0) {
      prev<N - 1>();
    }
  }
}
--it;
```

3825. Missing compile-time argument `id` check in `basic_format_parse_context::next_arg_id`

Section: 22.14.6.5 [format.parse.ctx] **Status:** Ready **Submitter:** Victor Zverovich **Opened:** 2022-11-09 **Last modified:** 2022-11-11

Priority: Not Prioritized

Discussion:

The definition of `check_arg_id` in 22.14.6.5 [format.parse.ctx] includes a (compile-time) argument `id` check in the *Remarks* element:

```
constexpr void check_arg_id(size_t id);
```

[…]

Remarks: Call expressions where `id >= num_args_` are not core constant expressions (7.7 [expr.const]).

However, a similar check is missing from `next_arg_id` which means that there is no argument id validation in user-defined format specification parsing code that invokes this function (e.g. when parsing automatically indexed dynamic width).

Previous resolution [SUPERSEDED]:

This wording is relative to N4917.

1. Modify 22.14.6.5 [format.parse.ctx] as indicated:

```
constexpr size_t next_arg_id();
```

-7- *Effects*: If `indexing_ != manual`, equivalent to:

```
if (indexing_ == unknown)
  indexing_ = automatic;
return next_arg_id_++;
```

-8- *Throws*: `format_error` if `indexing_ == manual` which indicates mixing of automatic and manual argument indexing.

-?- *Remarks*: Call expressions where `next_arg_id_ >= num_args_` are not core constant expressions (7.7 [expr.const]).

[2022-11-11; Tomasz provide improved wording; Move to Open]

Clarify that the value of `next_arg_id_` is used, and add missing "is `true`."

[Kona 2022-11-11; move to Ready]

Proposed resolution:

This wording is relative to N4917.

1. Modify 22.14.6.5 [format.parse.ctx] as indicated:

```
constexpr size_t next_arg_id();
```

-7- *Effects*: If `indexing_ != manual` is `true`, equivalent to:

```
if (indexing_ == unknown)
  indexing_ = automatic;
return next_arg_id_++;
```

-8- *Throws*: `format_error` if `indexing_ == manual` is `true` which indicates mixing of automatic and manual argument indexing.

-?- *Remarks*: Let *cur-arg-id* be the value of `next_arg_id_` prior to this call. Call expressions where *cur-arg-id* `>= num_args_` is `true` are not core constant expressions (7.7 [expr.const]).

```
constexpr size_t check_arg_id(size_t id);
```

-9- *Effects*: If `indexing_ != automatic` is `true`, equivalent to:

```
if (indexing_ == unknown)
  indexing_ = manual;
```

-10- *Throws*: `format_error` if `indexing_ == automatic` is `true` which indicates mixing of automatic and manual argument indexing.

-11- *Remarks*: Call expressions where `id >= num_args_` is `true` are not core constant expressions (7.7 [expr.const]).

Tentatively Ready Issues

3204. `sub_match::swap` only swaps the base class

Section: 32.8 [re.submatch] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2019-05-07 **Last modified:** 2022-11-30

Priority: 3

View other active issues in [re.submatch].

View all other issues in [re.submatch].

Discussion:

`sub_match<I>` derives publicly from `pair<I,I>`, and so inherits `pair::swap(pair&)`. This means that the following program fails:

```
#include <regex>
#include <cassert>

int main()
{
  std::sub_match<const char*> a, b;
  a.matched = true;
  a.swap(b);
  assert(b.matched);
}
```

The `pair::swap(pair&)` member should be hidden by a `sub_match::swap(sub_match&)` member that does the right thing.

[2019-06-12 Priority set to 3 after reflector discussion]

[2020-05-01; Daniel adjusts wording to recent working draft]

[2022-04-25; Daniel adjusts wording to recent working draft]

In addition the revised wording uses the new standard phrase "The exception specification is equivalent to"

Previous resolution [SUPERSEDED]:

This wording is relative to N4910.

1. Modify 32.8 [re.submatch], class template `sub_match` synopsis, as indicated:

```
template<class BidirectionalIterator>
class sub_match : public pair<BidirectionalIterator, BidirectionalIterator> {
public:
  […]
  int compare(const sub_match& s) const;
  int compare(const string_type& s) const;
  int compare(const value_type* s) const;

  void swap(sub_match& s) noexcept(see below);
};
```

2. Modify 32.8.2 [re.submatch.members] as indicated:

```
int compare(const value_type* s) const;
```

[…]

```
void swap(sub_match& s) noexcept(see below);
```

[*Drafting note:* The swappable requirement should really be unnecessary because *Cpp17Iterator* requires it, but there is no wording that requires `BidirectionalIterator` in Clause 32 [re] in general meets the bidirectional iterator requirements. Note that the definition found in 27.2 [algorithms.requirements] does not extend to 32 [re] normatively. — *end drafting note*]

-?- *Preconditions:* Lvalues of type `BidirectionalIterator` are swappable (16.4.4.3 [swappable.requirements]).

-?- *Effects:* Equivalent to:

```
this->pair<BidirectionalIterator, BidirectionalIterator>::swap(s);
std::swap(matched, s.matched);
```

-?- *Remarks:* The exception specification is equivalent to `is_nothrow_swappable_v<BidirectionalIterator>`.

[2022-11-06; Daniel comments and improves wording]

With the informal acceptance of P2696R0 by LWG during a pre-Kona telecon, we should use the new requirement set *Cpp17Swappable* instead of the "LValues are swappable" requirements.

[2022-11-30; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917 **and assumes the acceptance of P2696R0**.

1. Modify 32.8 [re.submatch], class template `sub_match` synopsis, as indicated:

```
template<class BidirectionalIterator>
class sub_match : public pair<BidirectionalIterator, BidirectionalIterator> {
public:
  […]
  int compare(const sub_match& s) const;
  int compare(const string_type& s) const;
  int compare(const value_type* s) const;

  void swap(sub_match& s) noexcept(see below);
};
```

2. Modify 32.8.2 [re.submatch.members] as indicated:

```
int compare(const value_type* s) const;
```

[…]

```
void swap(sub_match& s) noexcept(see below);
```

[*Drafting note:* The *Cpp17Swappable* requirement should really be unnecessary because *Cpp17Iterator* requires it, but there is no wording that requires `BidirectionalIterator` in Clause 32 [re] in general meets the bidirectional iterator requirements. Note that the definition found in 27.2 [algorithms.requirements] does not extend to 32 [re] normatively. — *end drafting note*]

-?- *Preconditions:* `BidirectionalIterator` meets the *Cpp17Swappable* requirements (16.4.4.3 [swappable.requirements]).

-?- *Effects:* Equivalent to:

```
this->pair<BidirectionalIterator, BidirectionalIterator>::swap(s);
std::swap(matched, s.matched);
```

-?- *Remarks:* The exception specification is equivalent to `is_nothrow_swappable_v<BidirectionalIterator>`.

3733. `ranges::to` misuses *`cpp17-input-iterator`*

Section: 26.5.7.2 [range.utility.conv.to] **Status:** Tentatively Ready **Submitter:** S. B. Tam **Opened:** 2022-07-10 **Last modified:** 2023-01-11

Priority: 2

View other active issues in [range.utility.conv.to].

View all other issues in [range.utility.conv.to].

Discussion:

`ranges::to` uses *`cpp17-input-iterator`*`<iterator_t<R>>` to check whether an iterator is a *Cpp17InputIterator*, which misbehaves if there is a `std::iterator_traits` specialization for that iterator (e.g. if the iterator is a `std::common_iterator`).

```
struct MyContainer {
    template<class Iter>
    MyContainer(Iter, Iter);

    char* begin();
    char* end();
};

auto nul_terminated = std::views::take_while([](char ch) { return ch != '\0'; });
auto c = nul_terminated("") | std::views::common | std::ranges::to<MyContainer>();  // error
```

I believe that `ranges::to` should instead use `derived_from<typename iterator_traits<iterator_t<R>>::iterator_category, input_iterator_tag>`, which correctly detects the iterator category of a `std::common_iterator`.

[2022-08-23; Reflector poll]

Set priority to 2 after reflector poll. Set status to Tentatively Ready after five votes in favour during reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to N4910.

1. Modify 26.5.7.2 [range.utility.conv.to] as indicated:

```
template<class C, input_range R, class... Args> requires (!view<C>)
  constexpr C to(R&& r, Args&&... args);
```

-1- *Returns*: An object of type `C` constructed from the elements of `r` in the following manner:

(1.1) — If `convertible_to<range_reference_t<R>, range_value_t<C>>` is `true`:

(1.1.1) — If `constructible_from<C, R, Args...>` is `true`:

```
C(std::forward<R>(r), std::forward<Args>(args)...)
```

(1.1.2) — Otherwise, if `constructible_from<C, from_range_t, R, Args...>` is `true`:

```
C(from_range, std::forward<R>(r), std::forward<Args>(args)...)
```

(1.1.3) — Otherwise, if

(1.1.3.1) — `common_range<R>` is `true`,

(1.1.3.2) — ~~*cpp17-input-iterator*~~`derived_from<typename iterator_traits<iterator_t<R>>::iterator_category, input_iterator_tag>` is `true`, and

(1.1.3.3) — `constructible_from<C, iterator_t<R>, sentinel_t<R>, Args...>` is `true`:

```
C(ranges::begin(r), ranges::end(r), std::forward<Args>(args)...)
```

(1.1.4) — Otherwise, if

(1.1.4.1) — `constructible_from<C, Args...>` is `true`, and

(1.1.4.2) — *`container-insertable`*`<C, range_reference_t<R>>` is `true`:

```
[…]
```

(1.2) — Otherwise, if `input_range<range_reference_t<R>>` is `true`:

```
to<C>(r | views::transform([](auto&& elem) {
  return to<range_value_t<C>>(std::forward<decltype(elem)>(elem));
}), std::forward<Args>(args)...);
```

(1.3) — Otherwise, the program is ill-formed.

[2022-08-27; Hewill Kang reopens and suggests a different resolution]

This issue points out that the standard misuses *`cpp17-input-iterator`* to check whether the iterator meets the requirements of *Cpp17InputIterator*, and proposes to use `iterator_traits<I>::iterator_category` to check the iterator's category directly, which may lead to the following potential problems:

First, for the range types that model both `common_range` and `input_range`, the expression `iterator_traits<I>::iterator_category` may not be valid, consider

```
#include <ranges>

struct I {
  using difference_type = int;
  using value_type = int;
  int operator*() const;
  I& operator++();
  void operator++(int);
  bool operator==(const I&) const;
  bool operator==(std::default_sentinel_t) const;
};

int main() {
  auto r = std::ranges::subrange(I{}, I{});
  auto v = r | std::ranges::to<std::vector<int>>(0);
}
```

Although `I` can serve as its own sentinel, it does not model *`cpp17-input-iterator`* since postfix `operator++` returns `void`, which causes `iterator_traits<R>` to be an empty class, making the expression `derived_from<iterator_traits<I>::iterator_category, input_iterator_tag>` ill-formed.

Second, for `common_iterator`, `iterator_traits<I>::iterator_category` does not guarantee a strictly correct iterator category in the current standard.

For example, when the above `I::operator*` returns a non-copyable object that can be converted to `int`, this makes `common_iterator<I, default_sentinel_t>` unable to synthesize a C++17-conforming postfix `operator++`, however, `iterator_traits<common_iterator<I, S>>::iterator_category` will still give `input_iterator_tag` even though it's not even a C++17 iterator.

Another example is that for `input_iterators` with difference type of integer-class type, the difference type of the `common_iterator` wrapped on it is still of the integer-class type, but the `iterator_category` obtained by the `iterator_traits` is `input_iterator_tag`.

The proposed resolution only addresses the first issue since I believe that the problem with `common_iterator` requires a paper.

[2023-01-11; LWG telecon]

Set status to Tentatively Ready (poll results F6/A0/N1)

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.5.7.2 [range.utility.conv.to] as indicated:

```
template<class C, input_range R, class... Args> requires (!view<C>)
  constexpr C to(R&& r, Args&&... args);
```

-1- *Returns*: An object of type `C` constructed from the elements of `r` in the following manner:

(1.1) — If `convertible_to<range_reference_t<R>, range_value_t<C>>` is `true`:

(1.1.1) — If `constructible_from<C, R, Args...>` is `true`:

```
C(std::forward<R>(r), std::forward<Args>(args)...)
```

(1.1.2) — Otherwise, if `constructible_from<C, from_range_t, R, Args...>` is `true`:

```
C(from_range, std::forward<R>(r), std::forward<Args>(args)...)
```

(1.1.3) — Otherwise, if

(1.1.3.1) — `common_range<R>` is `true`,

(1.1.3.2) — ~~*cpp17-input-iterator*~~if the *qualified-id* `iterator_traits<iterator_t<R>>::iterator_category` is ~~`true`~~ valid and denotes a type that models `derived_from<input_iterator_tag>`, and

(1.1.3.3) — `constructible_from<C, iterator_t<R>, sentinel_t<R>, Args...>`:

```
C(ranges::begin(r), ranges::end(r), std::forward<Args>(args)...)
```

(1.1.4) — Otherwise, if

(1.1.4.1) — `constructible_from<C, Args...>` is `true`, and

(1.1.4.2) — *`container-insertable`*`<C, range_reference_t<R>>` is `true`:

[…]

(1.2) — Otherwise, if `input_range<range_reference_t<R>>` is `true`:

```
to<C>(r | views::transform([](auto&& elem) {
  return to<range_value_t<C>>(std::forward<decltype(elem)>(elem));
}), std::forward<Args>(args)...);
```

(1.3) — Otherwise, the program is ill-formed.

3742. `deque::prepend_range` needs to permute

Section: 24.2.4 [sequence.reqmts] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2022-07-16 **Last modified:** 2022-11-30

Priority: 2

View other active issues in [sequence.reqmts].

View all other issues in [sequence.reqmts].

Discussion:

When the range to be inserted is neither bidirectional nor sized, it's simpler to prepend elements one at a time, and then reverse the prepended elements. When the range to be inserted is neither forward nor sized, I believe this approach is necessary to implement `prepend_range` at all — there is no way to determine the length of the range modulo the block size of the `deque` ahead of time so as to insert the new elements in the proper position.

The container requirements do not allow `prepend_range` to permute elements in a `deque`. I believe we *must* allow permutation when the range is neither forward nor sized, and we *should* allow permutation when the range is not bidirectional to allow implementations the freedom to make a single pass through the range.

[2022-07-17; Daniel comments]

The below suggested wording follows the existing style used in the specification of `insert` and `insert_range`, for example. Unfortunately, this existing practice violates the usual wording style that a *Cpp17XXX* requirement shall be *met* and that we should better say that "lvalues of type `T` are swappable (16.4.4.3 [swappable.requirements])" to be clearer about the specific swappable context. A separate editorial issue will be reported to take care of this problem.

[2022-08-23; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Previous resolution [SUPERSEDED]:

> This wording is relative to N4910.

1. Modify 24.2.4 [sequence.reqmts] as indicated:

`a.prepend_range(rg)`

-94- *Result:* `void`

-95- *Preconditions:* `T` is *Cpp17EmplaceConstructible* into `X` from `*ranges::begin(rg)`. For `deque`, `T` is also `Cpp17MoveInsertable` into `X`, *Cpp17MoveConstructible*, *Cpp17MoveAssignable*, and swappable (16.4.4.3 [swappable.requirements]).

-96- *Effects:* Inserts copies of elements in `rg` before `begin()`. Each iterator in the range `rg` is dereferenced exactly once.

[*Note 3*: The order of elements in `rg` is not reversed. — *end note*]

-97- *Remarks:* Required for `deque`, `forward_list`, and `list`.

[2022-11-07; Daniel reopens and comments]

The proposed wording has two problems:

1. It still uses "swappable" instead of "swappable lvalues", which with the (informal) acceptance of P2696R0 should now become *Cpp17Swappable*.
2. It uses an atypical form to say "`T` (is) *Cpp17MoveConstructible*, *Cpp17MoveAssignable*" instead of the more correct form "`T` meets the *Cpp17MoveConstructible* and *Cpp17MoveAssignable* requirements". This form was also corrected by P2696R0.

The revised wording uses the P2696R0 wording approach to fix both problems.

[Kona 2022-11-12; Set priority to 2]

[2022-11-30; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917 **assuming that P2696R0 has been accepted**.

1. Modify 24.2.4 [sequence.reqmts] as indicated:

`a.prepend_range(rg)`

-94- *Result:* `void`

-95- *Preconditions:* `T` is *Cpp17EmplaceConstructible* into `X` from `*ranges::begin(rg)`. For `deque`, `T` is also `Cpp17MoveInsertable` into `X`, and `T` meets the *Cpp17MoveConstructible*, *Cpp17MoveAssignable*, and *Cpp17Swappable* (16.4.4.3 [swappable.requirements]) requirements.

-96- *Effects:* Inserts copies of elements in `rg` before `begin()`. Each iterator in the range `rg` is dereferenced exactly once.

[*Note 3*: The order of elements in `rg` is not reversed. — *end note*]

-97- *Remarks:* Required for `deque`, `forward_list`, and `list`.

3790. P1467 accidentally changed `nexttoward`'s signature

Section: 28.7.1 [cmath.syn] **Status:** Tentatively Ready **Submitter:** David Olsen **Opened:** 2022-09-30 **Last modified:** 2023-01-11

Priority: 1

View other active issues in [cmath.syn].

View all other issues in [cmath.syn].

Discussion:

P1467 (Extended floating-point types), which was adopted for C++23 at the July plenary, has a typo (which is entirely my fault) that no one noticed during wording review. The changes to the `<cmath>` synopsis in the paper included changing this:

```
constexpr float nexttoward(float x, long double y);       // see [library.c]
constexpr double nexttoward(double x, long double y);
constexpr long double nexttoward(long double x, long double y);   // see [library.c]
```

to this:

```
constexpr floating-point-type nexttoward(floating-point-type x, floating-point-type y);
```

That changed the second parameter of `nexttoward` from always being `long double` to being *`floating-point-type`*, which matches the type of the first parameter.

The change is obviously incorrect. The purpose of the changes to `<cmath>` was to add overloads of the functions for extended floating-point types, not to change any existing signatures.

[2022-10-10; Reflector poll]

Set priority to 1 after reflector poll. Discussion during prioritization revolved around whether to delete `nexttoward` for new FP types or just restore the C++20 signatures, which might accept the new types via implicit conversions (and so return a different type, albeit with the same representation and same set of values).

"When the first argument to `nexttoward` is an extended floating-point type that doesn't have the same representation as a standard floating-point type, such as `std::float16_t`, `std::bfloat16_t`, or `std::float128_t` (on some systems), the call to `nexttoward` is ambiguous and ill-formed, so the unexpected return type is not an issue. Going through the extra effort of specifying '= delete' for `nexttoward` overloads that have extended floating-point arguments is a solution for a problem that doesn't really exist."

Previous resolution [SUPERSEDED]:

This wording is relative to N4917.

1. Modify 28.7.1 [cmath.syn], header <cmath> synopsis, as indicated:

```
[…]
constexpr floating-point-type nexttoward(floating-point-type x, ~~floating-point-type~~long double y);
constexpr float nexttowardf(float x, long double y);
constexpr long double nexttowardl(long double x, long double y);
[…]
```

[2022-10-04; David Olsen comments and provides improved wording]

C23 specifies variants of most of the functions in <math.h> for the `_FloatN` types (which are C23's equivalent of C++23's `std::floatN_t` types). But it does not specify those variants for `nexttoward`.

Based on what C23 is doing, I think it would be reasonable to leave `nexttoward`'s signature unchanged from C++20. There would be no requirement to provide overloads for extended floating-point types, only for the standard floating-point types. Instead of explicitly deleting the overloads with extended floating-point types, we can just never declare them in the first place.

Previous resolution [SUPERSEDED]:

This wording is relative to N4917.

1. Modify 28.7.1 [cmath.syn], header <cmath> synopsis, as indicated:

```
[…]
constexpr float nexttoward(float x, long double y);
constexpr double nexttoward(double x, long double y);
constexpr long double nexttoward(long double x, long double y);
~~constexpr floating-point-type nexttoward(floating-point-type x, floating-point-type y);~~
constexpr float nexttowardf(float x, long double y);
constexpr long double nexttowardl(long double x, long double y);
[…]
```

[2022-11-12; Tomasz comments and provides improved wording]

During 2022-10-26 LWG telecon we decided that we want to make the calls of the `nexttoward` to be ill-formed (equivalent of Mandates) when the first argument is extended floating-point type.

[2023-01-11; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 28.7.1 [cmath.syn], header <cmath> synopsis, as indicated:

```
[…]
constexpr floating-point-type nexttoward(floating-point-type x, ~~floating-point-type~~long double y);
constexpr float nexttowardf(float x, long double y);
constexpr long double nexttowardl(long double x, long double y);
[…]
```

2. Add following paragraph at the end of 28.7.1 [cmath.syn], header <cmath> synopsis:

-?- An invocation of `nexttoward` is ill-formed if the argument corresponding to the *floating-point-type* parameter has extended floating-point type.

3819. `reference_meows_from_temporary` should not use `is_meowible`

Section: 21.3.5.4 [meta.unary.prop] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2022-11-08 **Last modified:** 2022-11-10

Priority: Not Prioritized

View other active issues in [meta.unary.prop].

View all other issues in [meta.unary.prop].

Discussion:

The intent of P2255R2 is for the `reference_meows_from_temporary` traits to fully support cases where a prvalue is used as the source. Unfortunately the wording fails to do so because it tries to use the `is_meowible` traits to say "the initialization is well-formed", but those traits only consider initialization from xvalues, not prvalues. For example, given:

```
struct U {
  U();
  U(U&&) = delete;
};

struct T {
  T(U);
};
```

`reference_constructs_from_temporary_v<const T&, U>` should be true, but is currently defined as `false`. We need to spell out the "is well-formed" condition directly.

[Kona 2022-11-08; Move to Tentatively Ready]

Proposed resolution:

This wording is relative to N4917.

> [*Drafting note:* The note is already repeated every time we talk about "immediate context".]

1. Modify 21.3.3 [meta.type.synop], Table 46 ([tab:meta.unary.prop]) — "Type property predicates" — as indicated:

Table 46: Type property predicates [tab:meta.unary.prop]

Template	Condition	Preconditions
…		
`template<class T, class U> struct reference_constructs_from_temporary;`	~~`conjunction_v<is_reference<T>, is_constructible<T, U>>` is true~~`T` is a reference type, and the initialization `T t(VAL<U>);` is well-formed and binds `t` to a temporary object whose lifetime is extended (6.7.7 [class.temporary]). Access checking is performed as if in a context unrelated to `T` and `U`. Only the validity of the immediate context of the variable initialization is considered. [*Note ?:* The initialization can result in effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such effects are not in the "immediate context" and can result in the program being ill-formed. — *end note*]	`T` and `U` shall be complete types, *cv* `void`, or arrays of unknown bound.
`template<class T, class U> struct reference_converts_from_temporary;`	~~`conjunction_v<is_reference<T>, is_convertible<U, T>>` is true~~`T` is a reference type, and the initialization `T t = VAL<U>;` is well-formed and binds `t` to a temporary object whose lifetime is extended (6.7.7 [class.temporary]). Access checking is performed as if in a context unrelated to `T` and `U`. Only the validity of the immediate context of the variable initialization is considered. [*Note ?:* The initialization can result in effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such effects are not in the "immediate context" and can result in the program being ill-formed. — *end note*]	`T` and `U` shall be complete types, *cv* `void`, or arrays of unknown bound.

3821. `uses_allocator_construction_args` should have overload for *`pair-like`*

Section: 20.2.8.2 [allocator.uses.construction] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2022-11-08 **Last modified:** 2023-01-11

Priority: 2

View other active issues in [allocator.uses.construction].

View all other issues in [allocator.uses.construction].

Discussion:

P2165R4 added a *`pair-like`* constructor to `std::pair` but didn't add a corresponding `uses_allocator_construction_args` overload. It was in P2165R3 but incorrectly removed during the small group review.

Without LWG 3525, not having the overload would have caused emplacing a *`pair-like`* into a `pmr::vector<pair>` to be outright ill-formed.

With that issue's resolution, in cases where the constructor is not explicit we would create a temporary pair and then do uses-allocator construction using its pieces, and it still won't work when the constructor is explicit.

We should just do this properly.

[2022-11-09 Tim updates wording following LWG review]

During review of this issue LWG noticed that neither the constructor nor the new overload should accept `subrange`.

The `remove_cv_t` in the new paragraph is added for consistency with LWG 3677.

[Kona 2022-11-12; Set priority to 2]

[2023-01-11; LWG telecon]

Replace `P` with `U` in p17 and set status to Tentatively Ready (poll result: 8/0/0).

Proposed resolution:

This wording is relative to N4917 after the application of LWG 3677.

1. Edit 22.3.2 [pairs.pair] as indicated:

```
template<class U1, class U2> constexpr explicit(see below) pair(pair<U1, U2>& p);
template<class U1, class U2> constexpr explicit(see below) pair(const pair<U1, U2>& p);
template<class U1, class U2> constexpr explicit(see below) pair(pair<U1, U2>&& p);
template<class U1, class U2> constexpr explicit(see below) pair(const pair<U1, U2>&& p);
template<pair-like P> constexpr explicit(see below) pair(P&& p);
```

-14- Let *FWD*(u) be `static_cast<decltype(u)>(u)`.

-15- *Constraints:*

(15.?) — For the last overload, `remove_cvref_t<P>` is not a specialization of `ranges::subrange`,

(15.1) — `is_constructible_v<T1, decltype(get<0>(`*FWD*`(p)))>` is `true` and

(15.2) — `is_constructible_v<T2, decltype(get<1>(`*FWD*`(p)))>` is `true`.

-16- *Effects:* Initializes `first` with `get<0>(`*FWD*`(p))` and `second` with `get<1>(`*FWD*`(p))`.

2. Edit 20.2.2 [memory.syn], header `<memory>` synopsis, as indicated:

```
namespace std {
  […]
  // 20.2.8.2 [allocator.uses.construction], uses-allocator construction
  […]
  template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    pair<U, V>& pr) noexcept;

  template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    const pair<U, V>& pr) noexcept;

  template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    pair<U, V>&& pr) noexcept;

  template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    const pair<U, V>&& pr) noexcept;
  template<class T, class Alloc, pair-like P>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc, P&& p) noexcept;

  template<class T, class Alloc, class U>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc, U&& u) noexcept;
  […]
}
```

3. Add the following to 20.2.8.2 [allocator.uses.construction]:

```
  template<class T, class Alloc, pair-like P>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc, P&& p) noexcept;
```

-?- *Constraints:* `remove_cv_t<T>` is a specialization of `pair` and `remove_cvref_t<P>` is not a specialization of `ranges::subrange`.

-?- *Effects:* Equivalent to:

```
return uses_allocator_construction_args<T>(alloc, piecewise_construct,
                                           forward_as_tuple(get<0>(std::forward<P>(p))),
                                           forward_as_tuple(get<1>(std::forward<P>(p))));
```

4. Edit 20.2.8.2 [allocator.uses.construction] p17:

```
template<class T, class Alloc, class U>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc, U&& u) noexcept;
```

-16- Let *FUN* be the function template:

```
template<class A, class B>
void FUN(const pair<A, B>&);
```

-17- *Constraints:* `remove_cv_t<T>` is a specialization of `pair`, and either:

(17.1) — `remove_cvref_t<U>` is a specialization of `ranges::subrange`, or

(17.2) — `U` does not satisfy *pair-like* and the expression *FUN*`(u)` is not well-formed when considered as an unevaluated operand..

3834. Missing `constexpr` for `std::intmax_t` math functions in `<cinttypes>`

Section: 31.13.2 [cinttypes.syn] **Status:** Tentatively Ready **Submitter:** George Tokmaji **Opened:** 2022-11-27 **Last modified:** 2023-01-06

Priority: Not Prioritized

Discussion:

P0533R9 adds `constexpr` to math functions in `<cmath>` and `<cstdlib>`, which includes `std::abs` and `std::div`. This misses the overloads for `std::intmax_t` in `<cinttypes>`, as well as `std::imaxabs` and `std::imaxdiv`, which seems like an oversight.

[2023-01-06; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 31.13.2 [cinttypes.syn], header `<cinttypes>` synopsis, as indicated:

```
[…]
namespace std {
  using imaxdiv_t = see below;

  constexpr intmax_t imaxabs(intmax_t j);
  constexpr imaxdiv_t imaxdiv(intmax_t numer, intmax_t denom);
  intmax_t strtoimax(const char* nptr, char** endptr, int base);
  uintmax_t strtoumax(const char* nptr, char** endptr, int base);
  intmax_t wcstoimax(const wchar_t* nptr, wchar_t** endptr, int base);
  uintmax_t wcstoumax(const wchar_t* nptr, wchar_t** endptr, int base);

  constexpr intmax_t abs(intmax_t);            // optional, see below
  constexpr imaxdiv_t div(intmax_t, intmax_t); // optional, see below
  […]
}
[…]
```

-1- The contents and meaning of the header `<cinttypes>` are the same as the C standard library header `<inttypes.h>`, with the following changes:

(1.1) — The header `<cinttypes>` includes the header `<cstdint>` (17.4.1 [cstdint.syn]) instead of `<stdint.h>`, and

(1.2) — if and only if the type `intmax_t` designates an extended integer type (6.8.2 [basic.fundamental]), the following function signatures are added:

```
constexpr intmax_t abs(intmax_t);
constexpr imaxdiv_t div(intmax_t, intmax_t);
```

which shall have the same semantics as the function signatures `constexpr intmax_t imaxabs(intmax_t)` and `constexpr imaxdiv_t imaxdiv(intmax_t, intmax_t)`, respectively.

3839. `range_formatter`'s `set_separator`, `set_brackets`, and `underlying` functions should be `noexcept`

Section: 22.14.7.2 [format.range.formatter], 22.14.7.3 [format.range.fmtdef], 22.14.9 [format.tuple] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-12-11 **Last modified:** 2023-01-06

Priority: Not Prioritized

Discussion:

The `set_separator` and `set_brackets` of `range_formatter` only invoke `basic_string_view`'s assignment operator, which is `noexcept`, we should add `noexcept` specifications for them.

In addition, its `underlying` function returns a reference to the underlying `formatter`, which never throws, they should also be `noexcept`.

Similar rules apply to *`range-default-formatter`* and `formatter`'s `tuple` specialization.

[2023-01-06; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 22.14.7.2 [format.range.formatter] as indicated:

```
namespace std {
  template<class T, class charT = char>
    requires same_as<remove_cvref_t<T>, T> && formattable<T, charT>
  class range_formatter {
    formatter<T, charT> underlying_;                                    // exposition only
    basic_string_view<charT> separator_ = STATICALLY-WIDEN<charT>(", ");      // exposition only
    basic_string_view<charT> opening-bracket_ = STATICALLY-WIDEN<charT>("["); // exposition only
    basic_string_view<charT> closing-bracket_ = STATICALLY-WIDEN<charT>("]"); // exposition only

  public:
    constexpr void set_separator(basic_string_view<charT> sep) noexcept;
    constexpr void set_brackets(basic_string_view<charT> opening,
                                basic_string_view<charT> closing) noexcept;
    constexpr formatter<T, charT>& underlying() noexcept { return underlying_; }
```

```
    constexpr const formatter<T, charT>& underlying() const noexcept { return underlying_; }

    […]
  };
}
```

[…]

```
constexpr void set_separator(basic_string_view<charT> sep) noexcept;
```

-7- *Effects:* Equivalent to: `separator_ = sep;`

```
constexpr void set_brackets(basic_string_view<charT> opening, basic_string_view<charT> closing) noexcept;
```

-8- *Effects:* Equivalent to:

```
opening-bracket_ = opening;
closing-bracket_ = closing;
```

2. Modify 22.14.7.3 [format.range.fmtdef] as indicated:

```
namespace std {
  template<ranges::input_range R, class charT>
  struct range-default-formatter<range_format::sequence, R, charT> {    // exposition only
  private:
    using maybe-const-r = fmt-maybe-const<R, charT>;                    // exposition only
    range_formatter<remove_cvref_t<ranges::range_reference_t<maybe-const-r>>,
                    charT> underlying_;                                 // exposition only

  public:
    constexpr void set_separator(basic_string_view<charT> sep) noexcept;
    constexpr void set_brackets(basic_string_view<charT> opening,
                                basic_string_view<charT> closing) noexcept;

    […]
  };
}
```

```
constexpr void set_separator(basic_string_view<charT> sep) noexcept;
```

-1- *Effects:* Equivalent to: `underlying_.set_separator(sep);`.

```
constexpr void set_brackets(basic_string_view<charT> opening, basic_string_view<charT> closing) noexcept;
```

-2- *Effects:* Equivalent to: `underlying_.set_brackets(opening, closing);`.

3. Modify 22.14.9 [format.tuple] as indicated:

```
namespace std {
  template<class charT, formattable<charT>... Ts>
  struct formatter<pair-or-tuple<Ts...>, charT> {
  private:
    tuple<formatter<remove_cvref_t<Ts>, charT>...> underlying_;              // exposition only
    basic_string_view<charT> separator_ = STATICALLY-WIDEN<charT>(", ");     // exposition only
    basic_string_view<charT> opening-bracket_ = STATICALLY-WIDEN<charT>("("); // exposition only
    basic_string_view<charT> closing-bracket_ = STATICALLY-WIDEN<charT>(")"); // exposition only

  public:
    constexpr void set_separator(basic_string_view<charT> sep) noexcept;
    constexpr void set_brackets(basic_string_view<charT> opening,
                                basic_string_view<charT> closing) noexcept;

    […]
  };
}
```

[…]

```
constexpr void set_separator(basic_string_view<charT> sep) noexcept;
```

-5- *Effects:* Equivalent to: `separator_ = sep;`

```
constexpr void set_brackets(basic_string_view<charT> opening, basic_string_view<charT> closing) noexcept;
```

-6- *Effects:* Equivalent to:

```
opening-bracket_ = opening;
closing-bracket_ = closing;
```

3841. `<version>` should not be "*all freestanding*"

Section: 17.3.2 [version.syn] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2022-12-14 **Last modified:** 2023-01-06

Priority: Not Prioritized

View other active issues in [version.syn].

View all other issues in [version.syn].

Discussion:

It's reasonable for the `<version>` header to be required for freestanding, so that users can include it and see the "implementation-dependent information … (e.g. version number and release date)", and also to ask which features are present (which is the real intended purpose of `<version>`). It seems less reasonable to require every macro to be present on freestanding implementations, even the ones that correspond to non-freestanding features.

P2198R7 will fix this situation for C++26, but we should also do something for C++23 before publishing it. It seems sensible not to require any of the macros to be present, and then allow implementations to define them for the features that they support.

[2023-01-06; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 17.3.2 [version.syn], header `<version>` synopsis, as indicated:

> -2- Each of the macros defined in `<version>` is also defined after inclusion of any member of the set of library headers indicated in the corresponding comment in this synopsis.
>
> [*Note 1*: Future revisions of C++ might replace the values of these macros with greater values. — *end note*]

```
~~// all freestanding~~
#define __cpp_lib_addressof_constexpr          201603L // also in <memory>
[…]
```

3842. Unclear wording for *precision* in *chrono-format-spec*

Section: 29.12 [time.format] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2022-12-14 **Last modified:** 2023-01-06

Priority: Not Prioritized

View other active issues in [time.format].

View all other issues in [time.format].

Discussion:

29.12 [time.format] says:

> […] Giving a *precision* specification in the *chrono-format-spec* is valid only for `std::chrono::duration` types where the representation type `Rep` is a floating-point type. For all other `Rep` types, an exception of type `format_error` is thrown if the *chrono-format-spec* contains a *precision* specification. […]

It's unclear whether the restriction in the first sentence applies to all types, or only duration types. The second sentence seems to restrict the exceptional case to only types with a non-floating-point `Rep`, but what about types with no `Rep` type at all?

Can you use a precision with `sys_time<duration<float>>`? That is not a `duration` type at all, so does the restriction apply? What about `hh_mm_ss<duration<int>>`? That's not a `duration` type, but it uses one, and its `Rep` is not a floating-point type. What about `sys_info`? That's not a `duration` and doesn't have any associated `duration`, or `Rep` type.

What is the intention here?

Less importantly, I don't like the use of `Rep` here. That's the template parameter of the `duration` class template, but that name isn't in scope here. Why aren't we talking about the `duration` type's `rep` type, which is the public name for it? Or about a concrete specialization `duration<Rep, Period>`, instead of the class template?

The suggested change below would preserve the intended meaning, but with more … *precision*.

[2023-01-06; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 29.12 [time.format] as indicated:

> -1- […]
>
> The productions *fill-and-align*, *width*, and *precision* are described in 22.14.2 [format.string]. Giving a *precision* specification in the *chrono-format-spec* is valid only for types that are specializations of `std::chrono::duration` ~~types where the representation type `Rep` is~~for which the nested *typedef-name* `rep` denotes a floating-point type. For all other ~~`Rep`~~ types, an exception of type `format_error` is thrown if the *chrono-format-spec* contains a *precision* specification. […]

3848. `adjacent_view`, `adjacent_transform_view` and `slide_view` missing `base` accessor

Section: 26.7.25.2 [range.adjacent.view], 26.7.26.2 [range.adjacent.transform.view], 26.7.28.2 [range.slide.view] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2023-01-06 **Last modified:** 2023-02-01

Priority: Not Prioritized

Discussion:

Like most range adaptors, these three views accept a single range and store it as a member variable. However, they do not provide a `base()` member function for accessing the underlying range, which seems like an oversight.

[2023-02-01; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.7.25.2 [range.adjacent.view] as indicated:

```
namespace std::ranges {
  template<forward_range V, size_t N>
    requires view<V> && (N > 0)
  class adjacent_view : public view_interface<adjacent_view<V, N>> {
    V base_ = V();                      // exposition only
    […]
  public:
    adjacent_view() requires default_initializable<V> = default;
    constexpr explicit adjacent_view(V base);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }
    […]
  };
}
```

2. Modify 26.7.26.2 [range.adjacent.transform.view] as indicated:

```
namespace std::ranges {
  template<forward_range V, move_constructible F, size_t N>
    requires view<V> && (N > 0) && is_object_v<F> &&
             regular_invocable<F&, REPEAT(range_reference_t<V>, N)...> &&
             can-reference<invoke_result_t<F&, REPEAT(range_reference_t<V>, N)...>>
  class adjacent_transform_view : public view_interface<adjacent_transform_view<V, F, N>> {
    movable-box<F> fun_;                        // exposition only
    adjacent_view<V, N> inner_;                 // exposition only

    using InnerView = adjacent_view<V, N>;      // exposition only
    […]
  public:
    adjacent_transform_view() = default;
    constexpr explicit adjacent_transform_view(V base, F fun);

    constexpr V base() const & requires copy_constructible<InnerView> { return inner_.base(); }
    constexpr V base() && { return std::move(inner_).base(); }
    […]
  };
}
```

3. Modify 26.7.28.2 [range.slide.view] as indicated:

```
namespace std::ranges {
  […]
  template<forward_range V>
    requires view<V>
  class slide_view : public view_interface<slide_view<V>> {
    V base_;                            // exposition only
    range_difference_t<V> n_;           // exposition only
    […]
  public:
    constexpr explicit slide_view(V base, range_difference_t<V> n);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }
    […]
  };
  […]
}
```

3849. `cartesian_product_view::iterator`'s default constructor is overconstrained

Section: 99 [ranges.cartesian.iterator] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2023-01-06 **Last modified:** 2023-02-01

Priority: Not Prioritized

View other active issues in [ranges.cartesian.iterator].

View all other issues in [ranges.cartesian.iterator].

Discussion:

Currently, `cartesian_product_view::iterator` only provides the default constructor when the first range models `forward_range`, which seems too restrictive since several input iterators like `istream_iterator` are still default-constructible.

It would be more appropriate to constrain the default constructor only by whether the underlying iterator satisfies `default_initializable`, as most other range adaptors do. Since `cartesian_product_view::iterator` contains a `tuple` member that already has a constrained default constructor, the proposed resolution simply removes the constraint.

[2023-02-01; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 99 [ranges.cartesian.iterator] as indicated:

```
namespace std::ranges {
  template<input_range First, forward_range... Vs>
    requires (view<First> && ... && view<Vs>)
  template<bool Const>
  class cartesian_product_view<First, Vs...>::iterator {
  public:
    […]
    iterator() requires forward_range<maybe-const<Const, First>> = default;
    […]
  private:
    using Parent = maybe-const<Const, cartesian_product_view>;          // exposition only
    Parent* parent_ = nullptr;                                          // exposition only
    tuple<iterator_t<maybe-const<Const, First>>,
      iterator_t<maybe-const<Const, Vs>>...> current_;                  // exposition only
    […]
  };
}
```

3850. `views::as_const` on `empty_view<T>` should return `empty_view<const T>`

Section: 26.7.21.1 [range.as.const.overview] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2023-01-06 **Last modified:** 2023-02-01

Priority: Not Prioritized

View other active issues in [range.as.const.overview].

View all other issues in [range.as.const.overview].

Discussion:

Currently, applying `views::as_const` to an `empty_view<int>` will result in an `as_const_view<empty_view<int>>`, and its iterator type will be `basic_const_iterator<int*>`.

This amount of instantiation is not desirable for such a simple view, in which case simply returning `empty_view<const int>` should be enough.

[2023-02-01; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.7.21.1 [range.as.const.overview] as indicated:

> -2- The name `views::as_const` denotes a range adaptor object (26.7.2 [range.adaptor.object]). Let `E` be an expression, let `T` be `decltype((E))`, and let `U` be `remove_cvref_t<T>`. The expression `views::as_const(E)` is expression-equivalent to:
>
> (2.1) — If `views::all_t<T>` models `constant_range`, then `views::all(E)`.
>
> (2.?) — Otherwise, if `U` denotes `empty_view<X>` for some type `X`, then `auto(views::empty<const X>)`.
>
> (2.2) — Otherwise, if `U` denotes `span<X, Extent>` for some type `X` and some extent `Extent`, then `span<const X, Extent>(E)`.
>
> (2.3) — Otherwise, if `E` is an lvalue, `const U` models `constant_range`, and `U` does not model `view`, then `ref_view(static_cast<const U&>(E))`.
>
> (2.4) — Otherwise, `as_const_view(E)`.

3851. `chunk_view::inner-iterator` missing custom `iter_move` and `iter_swap`

Section: 26.7.27.5 [range.chunk.inner.iter] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2023-01-06 **Last modified:** 2023-02-01

Priority: Not Prioritized

Discussion:

For the input version of `chunk_view`, its *`inner-iterator`*`::operator*()` simply dereferences the underlying input iterator. However, there are no customized `iter_move` and `iter_swap` overloads for the *`inner-iterator`*, which prevents customized behavior when applying `views::chunk` to a range whose iterator type is a proxy iterator, for example:

```
#include <algorithm>
#include <cassert>
#include <ranges>
```

```
#include <sstream>
#include <vector>

int main() {
  auto ints = std::istringstream{"0 1 2 3 4"};
  std::vector<std::string> vs{"the", "quick", "brown", "fox"};
  auto r = std::views::zip(vs, std::views::istream<int>(ints))
         | std::views::chunk(2)
         | std::views::join;
  std::vector<std::tuple<std::string, int>> res;
  std::ranges::copy(std::move_iterator(r.begin()), std::move_sentinel(r.end()),
                    std::back_inserter(res));
  assert(vs.front().empty()); // assertion failed
}
```

`zip` iterator has a customized `iter_move` behavior, but since there is no `iter_move` specialization for *`inner-iterator`*, when we try to move elements in `chunk_view`, `move_iterator` will fallback to use the default implementation of `iter_move`, making `string`s not moved as expected from the original `vector` but copied instead.

[2023-02-01; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.7.27.5 [range.chunk.inner.iter] as indicated:

```
namespace std::ranges {
  template<view V>
    requires input_range<V>
  class chunk_view<V>::inner-iterator {
    chunk_view* parent_;                                              // exposition only

    constexpr explicit inner-iterator(chunk_view& parent) noexcept;   // exposition only

  public:
    […]
    friend constexpr difference_type operator-(default_sentinel_t y, const inner-iterator& x)
      requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
    friend constexpr difference_type operator-(const inner-iterator& x, default_sentinel_t y)
      requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;

    friend constexpr range_rvalue_reference_t<V> iter_move(const inner-iterator& i)
      noexcept(noexcept(ranges::iter_move(*i.parent_->current_)));

    friend constexpr void iter_swap(const inner-iterator& x, const inner-iterator& y)
      noexcept(noexcept(ranges::iter_swap(*x.parent_->current_, *y.parent_->current_)))
      requires indirectly_swappable<iterator_t<V>>;
  };
}
```

[…]

```
friend constexpr range_rvalue_reference_t<V> iter_move(const inner-iterator& i)
  noexcept(noexcept(ranges::iter_move(*i.parent_->current_)));
```

-?- *Effects:* Equivalent to: `return ranges::iter_move(*i.parent_->current_);`

```
friend constexpr void iter_swap(const inner-iterator& x, const inner-iterator& y)
  noexcept(noexcept(ranges::iter_swap(*x.parent_->current_, *y.parent_->current_)))
  requires indirectly_swappable<iterator_t<V>>;
```

-?- *Effects:* Equivalent to: `ranges::iter_swap(*x.parent_->current_, *y.parent_->current_);`

3853. `basic_const_iterator<volatile int*>::operator->` is ill-formed

Section: 25.5.3 [const.iterators] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2023-01-06 **Last modified:** 2023-02-01

Priority: Not Prioritized

View other active issues in [const.iterators].

View all other issues in [const.iterators].

Discussion:

Currently, `basic_const_iterator::operator->` constrains the value type of the underlying iterator to be only the *cv*-unqualified type of its reference type, which is true for raw pointers.

However, since it also explicitly specifies returning a pointer to a `const` value type, this will cause a hard error when the value type is actually `volatile`-qualified:

```
std::basic_const_iterator<volatile int*> it;
auto* p = it.operator->(); // invalid conversion from 'volatile int*' to 'const int*'
```

The proposed resolution changes the return type from `const value_type*` to `const auto*`, which makes it deduce the correct type in the above example, i.e. `const volatile int*`.

[2023-02-01; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 25.5.3 [const.iterators] as indicated:

```
namespace std {
  template<class I>
    concept not-a-const-iterator = see below;

  template<input_iterator Iterator>
  class basic_const_iterator {
    Iterator current_ = Iterator();                              // exposition only
    using reference = iter_const_reference_t<Iterator>;          // exposition only

  public:
    […]
    constexpr reference operator*() const;
    constexpr const autovalue_type* operator->() const
       requires is_lvalue_reference_v<iter_reference_t<Iterator>> &&
                same_as<remove_cvref_t<iter_reference_t<Iterator>>, value_type>;
    […]
  };
}

[…]

constexpr const autovalue_type* operator->() const
  requires is_lvalue_reference_v<iter_reference_t<Iterator>> &&
           same_as<remove_cvref_t<iter_reference_t<Iterator>>, value_type>;
```

-7- *Returns*: If `Iterator` models `contiguous_iterator`, `to_address(current_)`; otherwise, `addressof(*current_)`.

3857. `basic_string_view` should allow explicit conversion when only traits vary

Section: 23.3.3.2 [string.view.cons] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2023-01-10 **Last modified:** 2023-02-01

Priority: Not Prioritized

View all other issues in [string.view.cons].

Discussion:

`basic_string_view` has a constructor that converts appropriate contiguous ranges to `basic_string_view`. This constructor accepts an argument by forwarding reference (`R&&`), and has several constraints including that specified in 23.3.3.2 [string.view.cons]/12.6:

> if the *qualified-id* `remove_reference_t<R>::traits_type` is valid and denotes a type, `is_same_v<remove_reference_t<R>::traits_type, traits>` is `true`.

This constraint prevents conversions from `basic_string_view<C, T1>` and `basic_string<C, T1, A>` to `basic_string_view<C, T2>`. Preventing such seemingly semantic-affecting conversions from happening implicitly was a good idea, but since the constructor was changed to be explicit it no longer seems necessary to forbid these conversions. If a user wants to convert a `basic_string_view<C, T2>` to `basic_string_view<C, T1>` with `static_cast<basic_string_view<C, T1>>(meow)` instead of by writing out `basic_string_view<C, T1>{meow.data(), meow.size()}` that seems fine to me. Indeed, if we think conversions like this are so terribly dangerous we probably shouldn't be performing them ourselves in 22.14.8.1 [format.arg]/9 and 22.14.8.1 [format.arg]/10.

[2023-02-01; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 23.3.3.2 [string.view.cons] as indicated:

```
template<class R>
  constexpr explicit basic_string_view(R&& r);
```

-11- Let `d` be an lvalue of type `remove_cvref_t<R>`.

-12- *Constraints*:

(12.1) — `remove_cvref_t<R>` is not the same type as `basic_string_view`,

(12.2) — `R` models `ranges::contiguous_range` and `ranges::sized_range`,

(12.3) — `is_same_v<ranges::range_value_t<R>, charT>` is `true`,

(12.4) — `is_convertible_v<R, const charT*>` is `false`, and

(12.5) — `d.operator ::std::basic_string_view<charT, traits>()` is not a valid expression~~, and~~.

~~(12.6) — if the *qualified-id* `remove_reference_t<R>::traits_type` is valid and denotes a type, `is_same_v<remove_reference_t<R>::traits_type, traits>` is `true`.~~

3860. `range_common_reference_t` is missing

Section: 26.2 [ranges.syn] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2023-01-24 **Last modified:** 2023-02-06

Priority: Not Prioritized

View other active issues in [ranges.syn].

View all other issues in [ranges.syn].

Discussion:

For the alias template `iter_meow_t` in `<iterator>`, there are almost all corresponding `range_meow_t` in `<ranges>`, except for `iter_common_reference_t`, which is used to calculate the common reference type shared by `reference` and `value_type` of the iterator.

Given that it has a highly similar formula form to `iter_const_reference_t`, and the latter has a corresponding sibling, I think we should add a `range_common_reference_t` for `<ranges>`.

This increases the consistency of the two libraries and simplifies the text of getting common reference from a range. Since C++23 brings proxy iterators and `tuple` enhancements, I believe such introduction can bring some value.

[2023-02-06; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4928.

1. Modify 26.2 [ranges.syn], header `<ranges>` synopsis, as indicated:

```
#include <compare>              // see 17.11.1 [compare.syn]
#include <initializer_list>     // see 17.10.2 [initializer.list.syn]
#include <iterator>             // see 25.2 [iterator.synopsis]

namespace std::ranges {
  […]
  template<range R>
    using range_reference_t = iter_reference_t<iterator_t<R>>;                    // freestanding
  template<range R>
    using range_const_reference_t = iter_const_reference_t<iterator_t<R>>;        // freestanding
  template<range R>
    using range_rvalue_reference_t = iter_rvalue_reference_t<iterator_t<R>>;      // freestanding
  template<range R>
    using range_common_reference_t = iter_common_reference_t<iterator_t<R>>;      // freestanding
  […]
}
```

3866. Bad *Mandates* for `expected::transform_error` overloads

Section: 22.8.6.7 [expected.object.monadic], 22.8.7.7 [expected.void.monadic] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2023-01-29 **Last modified:** 2023-02-06

Priority: Not Prioritized

Discussion:

The overloads of `expected::transform_error` mandate that "[type] `G` is a valid value type for `expected`" (22.8.6.7 [expected.object.monadic]/27 and 31 as well as 22.8.7.7 [expected.void.monadic]/24 and 27)).

All of these overloads then instantiate `expected<T, G>` (for some type `T`) which doesn't require `G` to be a valid value type for `expected` (22.8.6.1 [expected.object.general]/2) but instead requires that `G` is "a valid template argument for `unexpected`" (22.8.6.1 [expected.object.general]/2). Comparing 22.8.6.1 [expected.object.general]/2 with 22.8.3.1 [expected.un.general]/2 it's clear that there are types — `const int`, for example — which are valid value types for `expected` but not valid template arguments for `unexpected`. Presumably this unimplementable requirement is a typo, and the subject paragraphs intended to require that `G` be a valid template argument for `unexpected`.

[2023-02-06; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4928.

1. Modify 22.8.6.7 [expected.object.monadic] as indicated:

```
template<class F> constexpr auto transform_error(F&& f) &;
template<class F> constexpr auto transform_error(F&& f) const &;
```

[…]

-27- *Mandates*: `G` is a valid ~~value type for `expected`~~template argument for `unexpected` (22.8.3.1 [expected.un.general]) and the declaration

```
G g(invoke(std::forward<F>(f), error()));
```

is well-formed.

[…]

[…]

```
template<class F> constexpr auto transform_error(F&& f) &&;
template<class F> constexpr auto transform_error(F&& f) const &&;
```

[…]

-31- *Mandates*: `G` is a valid ~~value type for `expected`~~template argument for `unexpected` (22.8.3.1 [expected.un.general]) and the declaration

```
G g(invoke(std::forward<F>(f), std::move(error())));
```

is well-formed.

[…]

2. Modify 22.8.7.7 [expected.void.monadic] as indicated:

```
template<class F> constexpr auto transform_error(F&& f) &;
template<class F> constexpr auto transform_error(F&& f) const &;
```

[…]

-24- *Mandates*: `G` is a valid ~~value type for `expected`~~template argument for `unexpected` (22.8.3.1 [expected.un.general]) and the declaration

```
G g(invoke(std::forward<F>(f), error()));
```

is well-formed.

[…]

[…]

```
template<class F> constexpr auto transform_error(F&& f) &&;
template<class F> constexpr auto transform_error(F&& f) const &&;
```

[…]

-27- *Mandates*: `G` is a valid ~~value type for `expected`~~template argument for `unexpected` (22.8.3.1 [expected.un.general]) and the declaration

```
G g(invoke(std::forward<F>(f), std::move(error())));
```

is well-formed.

[…]

3867. Should `std::basic_osyncstream`'s move assignment operator be `noexcept`?

Section: 31.11.3.1 [syncstream.osyncstream.overview] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2023-01-29 **Last modified:** 2023-02-06

Priority: Not Prioritized

View all other issues in [syncstream.osyncstream.overview].

Discussion:

The synopsis of `std::basic_osyncstream` (31.11.3.1 [syncstream.osyncstream.overview]) indicates that it's member functions behave as if it hold a `std::basic_syncbuf` as its subobject, and according to 16.3.3.4 [functions.within.classes], `std::basic_osyncstream`'s move assignment operator should call `std::basic_syncbuf`'s move assignment operator.

However, currently `std::basic_osyncstream`'s move assignment operator is `noexcept`, while `std::basic_syncbuf`'s is not. So when an exception is thrown from move assignment between `std::basic_syncbuf` objects, `std::terminate` should be called.

It's clarified in LWG 3498 that an exception can escape from `std::basic_syncbuf`'s move assignment operator. Is there any reason that an exception shouldn't escape from `std::basic_osyncstream`'s move assignment operator?

[2023-02-06; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4928.

1. Modify 31.11.3.1 [syncstream.osyncstream.overview] as indicated:

```
namespace std {
  template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
```

```
  class basic_osyncstream : public basic_ostream<charT, traits> {
  public:
    […]
    using syncbuf_type = basic_syncbuf<charT, traits, Allocator>;
    […]

    // assignment
    basic_osyncstream& operator=(basic_osyncstream&&) noexcept;

    […]

  private:
    syncbuf_type sb; // exposition only
  };
}
```